

LAPORAN PRAKTIKUM

STRUKTUR DATA

SORTING

PEKAN 8



Disusun Oleh :

KINAYA NOVRYA MANDA

(2511531016)

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis sampaikan kehadirat Allah SWT. Salawat dan salam disampaikan kepada Nabi Muhammad SAW. Karena thaufik dan hidayah-Nya, laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban dari kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar serta penerapan bahasa pemrograman Java.

Melalui praktikum ini, penulis memperoleh pengalaman langsung dalam memahami Implementasi Sorting menggunakan Java. Diharapkan laporan ini dapat memberikan gambaran yang jelas mengenai materi yang telah dipelajari serta hasil dari percobaan yang dilakukan selama praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi isi maupun penyajiannya. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu, asisten laboratorium, serta semua pihak yang telah memberikan bimbingan, arahan, dan dukungan sehingga laporan praktikum ini dapat tersusun. Semoga laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami dasar-dasar pemrograman Java.

Padang, Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB 1 PENDAHULUAN	
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	2
1.3 Manfaat Praktikum.....	2
BAB 2 PEMBAHASAN	
2.1 Dasar Teori.....	3
2.2 Kode Program.....	7
2.3 Langkah Kerja.....	15
2.4 Analisis Hasil.....	45
BAB 3 KESIMPULAN	
3.1 Hasil Praktikum.....	46
3.2 Saran Pengembangan.....	50
DAFTAR KEPUSTAKAAN.....	51
TUGAS PRAKTIKUM PEKAN 8.....	52

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, proses pengurutan data (Sorting) merupakan salah satu teknik yang sangat penting karena sering digunakan untuk mempermudah pencarian, pengolahan, dan penyajian informasi. Data yang tersusun dengan baik akan lebih mudah dianalisis dibandingkan data yang masih acak. Oleh karena itu, mahasiswa perlu memahami berbagai algoritma sorting beserta karakteristiknya. Pada praktikum ini dipelajari beberapa algoritma pengurutan, yaitu Shell Sort, Quick Sort, dan Merge Sort, yang masing-masing memiliki cara kerja dan tingkat efisiensi yang berbeda dalam mengurutkan data.

Selain memahami konsep pengurutan secara teori, praktikum juga memberikan kesempatan untuk mengimplementasikan algoritma tersebut dalam bentuk program Java. Melalui pembuatan program, mahasiswa dapat melihat secara langsung bagaimana proses pengurutan berlangsung dari data awal hingga menghasilkan data yang sudah terurut. Penggunaan beberapa algoritma yang berbeda memungkinkan mahasiswa membandingkan langkah kerja, struktur program, serta kelebihan dan kekurangan dari setiap metode sorting yang digunakan.

Tidak hanya itu, praktikum ini juga dilengkapi dengan pembuatan visualisasi menggunakan Bubble Sort GUI dan Merge Sort GUI. Visualisasi tersebut membantu memperjelas proses pengurutan karena setiap langkah ditampilkan secara bertahap melalui antarmuka grafis. Dengan adanya tampilan visual, proses perbandingan, pertukaran, dan penggabungan elemen dapat diamati dengan lebih mudah dibandingkan hanya melihat hasil akhir pada console. Melalui kegiatan praktikum ini, mahasiswa diharapkan tidak hanya mampu mengimplementasikan algoritma sorting, tetapi juga memahami cara kerja dan penerapannya dalam pengembangan program.

1.2 Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami konsep dasar algoritma sorting menggunakan metode Shell Sort, Quick Sort, dan MergeSort.
2. Mengetahui perbedaan proses pengurutan data pada masing-masing algoritma sorting yang digunakan dalam program.
3. Melatih kemampuan mahasiswa dalam membuat program pengurutan data serta menampilkan proses sorting melalui tampilan console dan GUI.

1.3 Manfaat Praktikum

Manfaat dilakukannya praktikum ini adalah sebagai berikut :

1. Mahasiswa dapat memahami cara kerja algoritma sorting secara lebih jelas melalui proses pengurutan data yang dilakukan langkah demi langkah.
2. Mahasiswa memperoleh pengalaman dalam mengimplementasikan algoritma sorting ke dalam bahasa pemrograman Java, baik berbasis console maupun GUI.
3. Praktikum membantu meningkatkan kemampuan logika pemrograman dan pemahaman terhadap pengolahan data menggunakan array dan proses perulangan.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Sorting merupakan proses mengurutkan data berdasarkan aturan tertentu, seperti urutan angka dari kecil ke besar atau data teks secara alfabetis dari A-Z. Dalam pemrograman, sorting digunakan untuk mempermudah pencarian, pengelompokan, dan pengolahan data sehingga informasi menjadi lebih terstruktur dan mudah dibaca. Proses sorting banyak digunakan dalam berbagai aplikasi komputer, misalnya pengurutan data mahasiswa, nilai, produk, maupun data pada database. Semakin teratur susunan data, maka proses pengolahan data juga akan menjadi lebih cepat dan efisien.

1. Shell Sort

Shell Sort merupakan algoritma pengurutan yang dikembangkan dari Insertion Sort dengan menggunakan konsep jarak atau gap dalam proses perbandingan data. Pada awal proses, elemen-elemen yang memiliki jarak tertentu dibandingkan dan diurutkan terlebih dahulu. Nilai gap kemudian diperkecil secara bertahap hingga bernilai satu, sehingga pada tahap akhir proses pengurutan berlangsung seperti Insertion Sort. Metode ini membuat data dapat berpindah lebih cepat ke posisi yang mendekati urutan yang benar dibandingkan algoritma pengurutan sederhana lainnya.

Pada program Shell Sort digunakan variabel `A_1016` untuk menyimpan data array yang akan diurutkan. Variabel `n_1016` digunakan untuk menyimpan jumlah elemen array, sedangkan `gap_1016` digunakan untuk menentukan jarak antar elemen yang dibandingkan selama proses sorting. Selain itu terdapat variabel `i_1016` sebagai indeks perulangan, `temp_1016` sebagai penyimpanan sementara, dan `j_1016` untuk mengatur proses pergeseran elemen sampai ditemukan posisi yang sesuai.

2. Quick Sort

Quick Sort merupakan algoritma pengurutan yang menggunakan metode divide and conquer, yaitu membagi data menjadi beberapa bagian yang lebih kecil berdasarkan sebuah nilai acuan yang disebut pivot. Data yang lebih kecil dari pivot ditempatkan di sebelah kiri, sedangkan data yang lebih besar ditempatkan di sebelah kanan. Proses ini dilakukan secara berulang hingga seluruh data berada dalam urutan yang benar.

Pada program Quick Sort digunakan array `arr_1016` sebagai tempat penyimpanan data yang akan diurutkan. Variabel `low_1016` dan `high_1016` digunakan untuk menentukan batas bagian array yang sedang diproses. Program juga menggunakan `pivot_1016` sebagai nilai acuan pengurutan, serta variabel `i_1016` dan `j_1016` untuk membantu proses pemisahan data. Selain itu terdapat method `swap_1016()` yang berfungsi menukar posisi dua elemen dan method `partition_1016()` yang digunakan untuk membagi data berdasarkan nilai pivot.

3. Merge Sort

Merge Sort merupakan algoritma pengurutan yang juga menggunakan metode divide and conquer. Algoritma ini bekerja dengan membagi array menjadi beberapa bagian yang lebih kecil hingga setiap bagian hanya memiliki satu elemen. Setelah itu, bagian-bagian tersebut digabungkan kembali sambil diurutkan sehingga menghasilkan array yang tersusun secara berurutan.

Pada program Merge Sort digunakan array `arr_1016` sebagai data utama yang akan diurutkan. Variabel `l_1016`, `m_1016`, dan `r_1016` digunakan untuk menentukan batas kiri, tengah, dan kanan dari array yang sedang diproses. Selain itu terdapat array sementara `L_1016` dan `R_1016` yang berfungsi menyimpan data selama proses penggabungan berlangsung. Variabel `i_1016`, `j_1016`, dan `k_1016` digunakan untuk membandingkan serta menyalin elemen ke posisi yang sesuai hingga seluruh data menjadi terurut.

4. Bubble Sort GUI

Bubble Sort GUI merupakan implementasi algoritma Bubble Sort dalam bentuk antarmuka grafis menggunakan Java Swing. Program ini memungkinkan pengguna memasukkan data array dan melihat proses pengurutan secara bertahap melalui tampilan visual. Setiap langkah perbandingan dan pertukaran elemen ditampilkan secara langsung sehingga pengguna dapat memahami proses kerja Bubble Sort dengan lebih mudah.

Pada program Bubble Sort GUI terdapat komponen seperti JFrame sebagai jendela utama, JPanel sebagai area tampilan array, JTextField untuk input data, JButton sebagai tombol kontrol, JLabel untuk menampilkan elemen array, dan JTextArea untuk menampilkan log proses sorting. Variabel `array_1016` digunakan untuk menyimpan data, `labelArray_1016` untuk menampilkan elemen array secara visual, sedangkan variabel `i_1016`, `j_1016`, `sorting_1016`, dan `stepCount_1016` digunakan untuk mengatur jalannya proses Bubble Sort langkah demi langkah.

5. Merge Sort GUI

Merge Sort GUI merupakan pengembangan algoritma Merge Sort yang ditampilkan dalam bentuk antarmuka grafis menggunakan Java Swing. Program ini tidak hanya menampilkan hasil akhir pengurutan, tetapi juga memperlihatkan setiap tahapan proses merge secara bertahap. Dengan adanya visualisasi ini, pengguna dapat melihat bagaimana data dibagi, dibandingkan, disalin ke array sementara, dan digabungkan kembali hingga menjadi urutan yang benar.

Pada program Merge Sort GUI digunakan berbagai komponen seperti JFrame, JPanel, JButton, JTextField, JLabel, dan JTextArea untuk membangun tampilan aplikasi. Variabel `array_1016` digunakan untuk menyimpan data yang akan diurutkan, `labelArray_1016` digunakan untuk menampilkan elemen array pada GUI, sedangkan `mergeQueue_1016` berfungsi menyimpan urutan proses

merge yang akan dijalankan. Selain itu terdapat variabel `left_1016`, `mid_1016`, `right_1016`, `temp_1016`, `i_1016`, `j_1016`, `k_1016`, `copying_1016`, dan `isMerging_1016` yang digunakan untuk mengatur proses penggabungan data secara bertahap hingga seluruh array tersusun secara terurut.

2.2 Kode Program

2.2.1 ShellSort_2511531016

```
package pekan8_2511531016;

public class ShellSort_2511531016 {
    public static void shellSort_1016(int[] A_1016) {
        int n_1016 = A_1016.length;
        int gap_1016 = n_1016 / 2;
        while (gap_1016 > 0) {
            for (int i_1016 = gap_1016; i_1016 < n_1016; i_1016++) {
                int temp_1016 = A_1016[i_1016];
                int j_1016 = i_1016;
                while (j_1016 >= gap_1016 && A_1016[j_1016 - gap_1016] > temp_1016) {
                    A_1016[j_1016] = A_1016[j_1016 - gap_1016];
                    j_1016 = j_1016 - gap_1016;
                }
                A_1016[j_1016] = temp_1016;
            }
            gap_1016 = gap_1016 / 2;
        }
    }

    public static void main(String[] args) {
        int[] data_1016 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
        System.out.print("Sebelum: ");
        printArray_1016(data_1016);

        shellSort_1016(data_1016);

        System.out.print("Sesudah (Shell Sort): ");
        printArray_1016(data_1016);
    }

    public static void printArray_1016(int[] arr_1016) {
        for (int i_1016 : arr_1016) System.out.print(i_1016 + " ");
        System.out.println();
    }
}
```

Kode Program 2.1 (Kode Program ShellSort)

2.2.2 QuickSort_2511531016

```
package pekan8_2511531016;

public class QuickSort_2511531016 {
    static void swap_1016(int[] arr_1016, int i_1016, int j_1016) {
        int temp_1016 = arr_1016[i_1016];
        arr_1016[i_1016] = arr_1016[j_1016];
        arr_1016[j_1016] = temp_1016;
    }

    // Metode tambahan untuk mengatur pivot menggunakan Median-of-Three
    static void medianOfThree_1016(int[] arr_1016, int low_1016, int high_1016) {
        int mid_1016 = low_1016 + (high_1016 - low_1016) / 2;

        // Urutkan elemen low, mid, dan high
        if (arr_1016[low_1016] > arr_1016[mid_1016]) {
            swap_1016(arr_1016, low_1016, mid_1016);
        }
        if (arr_1016[low_1016] > arr_1016[high_1016]) {
            swap_1016(arr_1016, low_1016, high_1016);
        }
        if (arr_1016[mid_1016] > arr_1016[high_1016]) {
            swap_1016(arr_1016, mid_1016, high_1016);
        }
        swap_1016(arr_1016, mid_1016, high_1016);
    }

    static int partition_1016(int[] arr_1016, int low_1016, int high_1016) {
        // Panggil fungsi medianOfThree sebelum menentukan pivot
        medianOfThree_1016(arr_1016, low_1016, high_1016);
        int pivot_1016 = arr_1016[high_1016]; // Sekarang arr[high] sudah berisi nilai median
        int i_1016 = (low_1016 - 1);

        for (int j_1016 = low_1016; j_1016 <= high_1016 - 1; j_1016++) {
            // Jika elemen saat ini lebih kecil dari atau sama dengan pivot
            if (arr_1016[j_1016] < pivot_1016) {

```

```

        // Increment indeks elemen yang lebih kecil
        i_1016++;
        swap_1016(arr_1016, i_1016, j_1016);
    }
    swap_1016(arr_1016, i_1016 + 1, high_1016);
    return (i_1016 + 1);
}

static void quickSort_1016(int[] arr_1016, int low_1016, int high_1016) {
    if (low_1016 < high_1016) {
        int pi_1016 = partition_1016(arr_1016, low_1016, high_1016);
        quickSort_1016(arr_1016, low_1016, pi_1016 - 1);
        quickSort_1016(arr_1016, pi_1016 + 1, high_1016);
    }
}

public static void printArr_1016(int[] arr_1016) {
    for (int i_1016 = 0; i_1016 < arr_1016.length; i_1016++) {
        System.out.print(arr_1016[i_1016] + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    int[] arr_1016 = {10, 7, 8, 9, 1, 5};
    int N_1016 = arr_1016.length;
    System.out.print("data sebelum diurutkan: ");
    printArr_1016(arr_1016);

    quickSort_1016(arr_1016, 0, N_1016 - 1);

    System.out.print("data terurut quicksort: ");
    printArr_1016(arr_1016);
}

```

Kode Program 2.2 (Kode Program QuickSort)

2.2.3 MergeSort 2511531016

```

package pekan8_2511531016;

public class MergeSort_2511531016 {
    void merge_1016(int arr_1016[], int l_1016, int m_1016, int r_1016) {
        // Find sizes of two subarrays to be merged
        int n1_1016 = m_1016 - l_1016 + 1;
        int n2_1016 = r_1016 - m_1016;

        /* Create temp arrays */
        int L_1016[] = new int[n1_1016];
        int R_1016[] = new int[n2_1016];

        /* Copy data to temp arrays */
        for (int i_1016 = 0; i_1016 < n1_1016; ++i_1016)
            L_1016[i_1016] = arr_1016[l_1016 + i_1016];

        for (int j_1016 = 0; j_1016 < n2_1016; ++j_1016)
            R_1016[j_1016] = arr_1016[m_1016 + 1 + j_1016];

        int i_1016 = 0, j_1016 = 0;

        // Initial index of merged subarray array
        int k_1016 = l_1016;

        while (i_1016 < n1_1016 && j_1016 < n2_1016) {
            if (L_1016[i_1016] <= R_1016[j_1016]) {
                arr_1016[k_1016] = L_1016[i_1016];
                i_1016++;
            } else {
                arr_1016[k_1016] = R_1016[j_1016];
                j_1016++;
            }
            k_1016++;
        }
    }
}

```

```

        /* Copy remaining elements of L[] if any */
        while (i_1016 < n1_1016) {
            arr_1016[k_1016] = L_1016[i_1016];
            i_1016++;
            k_1016++;
        }

        /* Copy remaining elements of R[] if any */
        while (j_1016 < n2_1016) {
            arr_1016[k_1016] = R_1016[j_1016];
            j_1016++;
            k_1016++;
        }
    }

    void sort_1016(int arr_1016[], int l_1016, int r_1016) {
        if (l_1016 < r_1016) {
            // Find the middle point
            int m_1016 = (l_1016 + r_1016) / 2;
            // Sort first and second halves
            sort_1016(arr_1016, l_1016, m_1016);
            sort_1016(arr_1016, m_1016 + 1, r_1016);
            // Merge the sorted halves
            merge_1016(arr_1016, l_1016, m_1016, r_1016);
        }
    }

    /* A utility function to print array of size n */
    static void printArray_1016(int arr_1016[]) {
        int n_1016 = arr_1016.length;
        for (int i_1016 = 0; i_1016 < n_1016; ++i_1016)
            System.out.print(arr_1016[i_1016] + " ");
        System.out.println();
    }

    public static void main(String args[]) {
        int arr_1016[] = {12, 11, 13, 5, 6, 7};
        System.out.println("Sebelum terurut");
        printArray_1016(arr_1016);

        MergeSort_2511531016 ob_1016 = new MergeSort_2511531016();
        ob_1016.sort_1016(arr_1016, 0, arr_1016.length - 1);

        System.out.println("\nSesudah Terurut menggunakan merge Sort");
        printArray_1016(arr_1016);
    }
}

```

Kode Program 2.3 (Kode Program MergeSort)

2.2.4 BubbleSortGUI_2511531016

```

package pekan8_2511531016;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextArea;
import javax.swing.border.EmptyBorder;

import pekan8_2511531016.BubbleSortGUI_2511531016;

public class BubbleSortGUI_2511531016 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane_1016;
    private int[] array_1016;
    private JLabel[] labelArray_1016;
    private JButton stepButton_1016, resetButton_1016, setButton_1016;
    private JTextField inputField_1016;
    private JPanel panelArray_1016;
    private JTextArea stanArea_1016;
}

```

```

private int i_1016 = 1, j_1016;
private boolean sorting_1016 = false;
private int stepCount_1016 = 1;

/**
 * Create the frame.
 */
public BubbleSortGUI_2511531016() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 450, 300);
    contentPane_1016 = new JPanel();
    contentPane_1016.setBorder(new EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane_1016);

    setTitle("Bubble Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    // Panel input
    JPanel inputPanel_1016 = new JPanel(new FlowLayout());
    inputField_1016 = new JTextField(30);
    setButton_1016 = new JButton("Set Array");
    inputPanel_1016.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
    inputPanel_1016.add(inputField_1016);
    inputPanel_1016.add(setButton_1016);

    // Panel array visual
    panelArray_1016 = new JPanel();
    panelArray_1016.setLayout(new FlowLayout());

    // Panel kontrol

    JPanel controlPanel_1016 = new JPanel();
    stepButton_1016 = new JButton("Langkah Selanjutnya");
    resetButton_1016 = new JButton("Reset");
    stepButton_1016.setEnabled(false);
    controlPanel_1016.add(stepButton_1016);
    controlPanel_1016.add(resetButton_1016);

    // Area teks untuk log langkah-langkah
    stepArea_1016 = new JTextArea(8, 60);
    stepArea_1016.setEditable(false);
    stepArea_1016.setFont(new Font("Monospaced", Font.PLAIN, 14));
    JScrollPane scrollPane_1016 = new JScrollPane(stepArea_1016);

    // Tambahkan panel ke frame
    add(inputPanel_1016, BorderLayout.NORTH);
    add(panelArray_1016, BorderLayout.CENTER);
    add(controlPanel_1016, BorderLayout.SOUTH);
    add(scrollPane_1016, BorderLayout.EAST);

    // Event Set Array
    setButton_1016.addActionListener(e -> setArrayFromInput_1016());

    // Event Langkah Selanjutnya
    stepButton_1016.addActionListener(e -> performStep_1016());

    // Event Reset
    resetButton_1016.addActionListener(e -> reset_1016());
}

private void setArrayFromInput_1016() {
    String text_1016 = inputField_1016.getText().trim();
    if (text_1016.isEmpty()) return;
    String[] parts_1016 = text_1016.split(",");

    array_1016 = new int[parts_1016.length];
    try {
        for (int k_1016 = 0; k_1016 < parts_1016.length; k_1016++) {
            array_1016[k_1016] = Integer.parseInt(parts_1016[k_1016].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka " + "yang dipisahkan koma!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    i_1016 = 0;
    j_1016 = 0;
    stepCount_1016 = 1;
    sorting_1016 = true;
    stepButton_1016.setEnabled(true);
    stepArea_1016.setText("");
    panelArray_1016.removeAll();
    labelArray_1016 = new JLabel[array_1016.length];
    for (int k_1016 = 0; k_1016 < array_1016.length; k_1016++) {
        labelArray_1016[k_1016] = new JLabel(String.valueOf(array_1016[k_1016]));
        labelArray_1016[k_1016].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_1016[k_1016].setOpaque(true);
        labelArray_1016[k_1016].setBackground(Color.WHITE);
        labelArray_1016[k_1016].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_1016[k_1016].setPreferredSize(new Dimension(50, 50));
        labelArray_1016[k_1016].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_1016.add(labelArray_1016[k_1016]);
    }
    panelArray_1016.revalidate();
    panelArray_1016.repaint();
}

private void performStep_1016() {
    if (!sorting_1016 || i_1016 >= array_1016.length - 1) {
        sorting_1016 = false;
    }
}

```



```

        stepButton_1016.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
        return;
    }
    resetHighlights();
    StringBuilder stepLog = new StringBuilder();
    labelArray_1016[j_1016].setBackground(Color.CYAN);
    labelArray_1016[j_1016 + 1].setBackground(Color.CYAN);
    if (array_1016[j_1016] > array_1016[j_1016 + 1]) {
        // swap
        int temp_1016 = array_1016[j_1016];
        array_1016[j_1016] = array_1016[j_1016 + 1];
        array_1016[j_1016 + 1] = temp_1016;
        labelArray_1016[j_1016].setBackground(Color.RED);
        labelArray_1016[j_1016 + 1].setBackground(Color.RED);

        stepLog.append("Langkah ").append(stepCount_1016)
            .append(": Menukar elemen ke-")
            .append(j_1016).append(" [")
            .append(array_1016[j_1016 + 1]).append("] dengan ke-")
            .append(j_1016 + 1).append(" [")
            .append(array_1016[j_1016]).append("]\n");
    } else {
        stepLog.append("Langkah ").append(stepCount_1016)
            .append(": Tidak ada pertukaran antara ke-")
            .append(j_1016).append(" dan ke-")
            .append(j_1016 + 1).append("\n");
    }

    stepLog.append("Hasil: ")
        .append(arrayToString_1016(array_1016))
        .append("\n\n");
    stepArea_1016.append(stepLog.toString());
    updateLabels_1016();

    j_1016++;

    if (j_1016 >= array_1016.length - i_1016 - 1) {
        j_1016 = 0;
        i_1016++;
    }
    stepCount_1016++;

    if (i_1016 >= array_1016.length - 1) {
        sorting_1016 = false;
        stepButton_1016.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}

private void updateLabels_1016() {
    for (int k_1016 = 0; k_1016 < array_1016.length; k_1016++) {
        labelArray_1016[k_1016].setText(String.valueOf(array_1016[k_1016]));
    }
}

private void resetHighlights() {
    for (JLabel label_1016 : labelArray_1016) {
        label_1016.setBackground(Color.WHITE);
    }
}

private void reset_1016() {
    inputField_1016.setText("");
    panelArray_1016.removeAll();
    panelArray_1016.revalidate();
    panelArray_1016.repaint();
    stepArea_1016.setText("");
    stepButton_1016.setEnabled(false);

    stepButton_1016.setEnabled(false);
    sorting_1016 = false;
    i_1016 = 0;
    j_1016 = 0;
    stepCount_1016 = 1;
}

private String arrayToString_1016(int[] arr_1016) {
    StringBuilder sb_1016 = new StringBuilder();
    for (int k_1016 = 0; k_1016 < arr_1016.length; k_1016++) {
        sb_1016.append(arr_1016[k_1016]);
        if (k_1016 < arr_1016.length - 1) sb_1016.append(", ");
    }
    return sb_1016.toString();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        BubbleSortGUI_2511531016 gui_1016 = new BubbleSortGUI_2511531016();
        gui_1016.setVisible(true);
    });
}
}

```

Kode Program 2.4 (Kode Program BubbleSortGUI)

2.2.5 MergeSortGUI_2511531016

```
package pekan8_2511531016;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;
import java.util.Queue;
import java.util.LinkedList;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextArea;
import javax.swing.border.EmptyBorder;

public class MergeSortGUI_2511531016 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane_1016;
    private int[] array_1016;
    private JLabel[] labelArray_1016;
    private JButton stepButton_1016, resetButton_1016, setButton_1016;
    private JTextField inputField_1016;
    private JPanel panelArray_1016;
    private JTextArea stepArea_1016;

    private boolean sorting_1016 = false;
    private boolean copying_1016 = false;
    private boolean isMerging_1016 = false;

    private int stepCount_1016 = 1;
    private java.util.Queue<int[]> mergeQueue_1016 = new java.util.LinkedList<>();
    private int left_1016, mid_1016, right_1016;
    private int[] temp_1016;
    private int i_1016, j_1016, k_1016;

    /**
     * Create the frame.
     */
    public MergeSortGUI_2511531016() {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 450, 300);
        contentPane_1016 = new JPanel();
        contentPane_1016.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane_1016);

        setTitle("Merge Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_1016 = new JPanel(new FlowLayout());
        inputField_1016 = new JTextField(30);
        setButton_1016 = new JButton("Set Array");
        inputPanel_1016.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
        inputPanel_1016.add(inputField_1016);
        inputPanel_1016.add(setButton_1016);

        // Panel array visual
        panelArray_1016 = new JPanel();
        panelArray_1016.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel_1016 = new JPanel();
        stepButton_1016 = new JButton("Langkah Selanjutnya");
        resetButton_1016 = new JButton("Reset");
        stepButton_1016.setEnabled(false);
        controlPanel_1016.add(stepButton_1016);
        controlPanel_1016.add(resetButton_1016);

        // Area teks untuk log langkah-langkah
        stepArea_1016 = new JTextArea(8, 60);
        stepArea_1016.setEditable(false);
        stepArea_1016.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane_1016 = new JScrollPane(stepArea_1016);

        // Tambahkan panel ke frame
        add(inputPanel_1016, BorderLayout.NORTH);
        add(panelArray_1016, BorderLayout.CENTER);
        add(controlPanel_1016, BorderLayout.SOUTH);
        add(scrollPane_1016, BorderLayout.EAST);

        // Event Set Array
        setButton_1016.addActionListener(e -> setArrayFromInput_1016());

        // Event Langkah Selanjutnya
        stepButton_1016.addActionListener(e -> performStep_1016());

        // Event Reset
        resetButton_1016.addActionListener(e -> reset_1016());
    }
}
```

```

private void setArrayFromInput_1016() {
    String text = inputField_1016.getText().trim();
    if (text.isEmpty()) return;
    String[] parts = text.split(",");
    array_1016 = new int[parts.length];
    try {
        for (int i = 0; i < parts.length; i++) {
            array_1016[i] = Integer.parseInt(parts[i].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    labelArray_1016 = new JLabel[array_1016.length];
    panelArray_1016.removeAll();

    for (int i = 0; i < array_1016.length; i++) {
        labelArray_1016[i] = new JLabel(String.valueOf(array_1016[i]));
        labelArray_1016[i].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_1016[i].setOpaque(true);
        labelArray_1016[i].setBackground(Color.WHITE);
        labelArray_1016[i].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_1016[i].setPreferredSize(new Dimension(50, 50));
        labelArray_1016[i].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_1016.add(labelArray_1016[i]);
    }
    mergeQueue_1016.clear();
    generateMergeSteps_1016(0, array_1016.length - 1);

    stepButton_1016.setEnabled(true);
    stepArea_1016.setText("");
    stepCount_1016 = 1;
    isMerging_1016 = false;

    panelArray_1016.revalidate();
    panelArray_1016.repaint();
}

private void generateMergeSteps_1016(int left_1016, int right_1016) {
    if (left_1016 >= right_1016) {
        return;
    }
    int mid_1016 = (left_1016 + right_1016) / 2;

    generateMergeSteps_1016(left_1016, mid_1016);
    generateMergeSteps_1016(mid_1016 + 1, right_1016);

    mergeQueue_1016.add(new int[] {
        left_1016, mid_1016, right_1016
    });
}

private void performStep_1016() {
    resetHighlights_1016();
    if (!isMerging_1016 && !mergeQueue_1016.isEmpty()) {
        int[] range_1016 = mergeQueue_1016.poll();

        left_1016 = range_1016[0];
        mid_1016 = range_1016[1];
        right_1016 = range_1016[2];
        temp_1016 = new int[right_1016 - left_1016 + 1];

        i_1016 = left_1016;
        j_1016 = mid_1016 + 1;
        k_1016 = 0;

        copying_1016 = false;
        isMerging_1016 = true;

        stepArea_1016.append("Langkah " + stepCount_1016++ + ": Mulai merge dari indeks " + left_1016 + " sampai " + right_1016 + "\n");
        return;
    }

    if (isMerging_1016 && !copying_1016) {
        if (i_1016 <= mid_1016 && j_1016 <= right_1016) {
            labelArray_1016[i_1016].setBackground(Color.CYAN);
            labelArray_1016[j_1016].setBackground(Color.CYAN);
            if (array_1016[i_1016] <= array_1016[j_1016]) {
                temp_1016[k_1016++] = array_1016[i_1016++];
            } else {
                temp_1016[k_1016++] = array_1016[j_1016++];
            }
        }
        stepArea_1016.append(
            "Langkah " + stepCount_1016++
            + ": Bandingkan dan salin elemen\n"
            + "Hasil: "
            + arrayToString_1016(array_1016)
            + "\n\n"
        );
        return;
    }
    else if (i_1016 <= mid_1016) {
        temp_1016[k_1016++] = array_1016[i_1016++];
        stepArea_1016.append(
            "Langkah " + stepCount_1016++
            + ": Salin sisa kiri\n"
            + "Hasil: "
            + arrayToString_1016(array_1016)
            + "\n\n"
        );
        return;
    }
    else if (j_1016 <= right_1016) {

```



```

        temp_1016[k_1016++] = array_1016[j_1016++];
        stepArea_1016.append(
            "Langkah " + stepCount_1016++
            + ": Salin sisa kanan\n"
            + "Hasil: "
            + arrayToString_1016(array_1016)
            + "\n\n"
        );
        return;
    }
    else {
        copying_1016 = true;
        k_1016 = 0;
        return;
    }
}

if (copying_1016 && k_1016 < temp_1016.length) {
    array_1016[left_1016 + k_1016] = temp_1016[k_1016];
    labelArray_1016[left_1016 + k_1016].setText(String.valueOf(temp_1016[k_1016]));
    labelArray_1016[left_1016 + k_1016].setBackground(Color.GREEN);
    k_1016++;
    stepArea_1016.append(
        "Langkah " + stepCount_1016++
        + ": Tempelkan ke array utama\n"
        + "Hasil: "
        + arrayToString_1016(array_1016)
        + "\n\n"
    );
    return;
}

if (copying_1016 && k_1016 == temp_1016.length) {
    isMerging_1016 = false;
    copying_1016 = false;

    if (mergeQueue_1016.isEmpty() && !isMerging_1016) {
        stepArea_1016.append("Selesai.\n");
        stepButton_1016.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
    }
}

private void resetHighlights_1016() {
    if (labelArray_1016 == null) {
        return;
    }
    for (JLabel label_1016 : labelArray_1016) {
        label_1016.setBackground(Color.WHITE);
    }
}

private void reset_1016() {
    inputField_1016.setText("");
    panelArray_1016.removeAll();
    panelArray_1016.revalidate();
    panelArray_1016.repaint();
    stepArea_1016.setText("");
    stepButton_1016.setEnabled(false);
    mergeQueue_1016.clear();
    isMerging_1016 = false;
    stepCount_1016 = 1;
}

private String arrayToString_1016(int[] arr_1016) {
    StringBuilder sb_1016 = new StringBuilder();
    for (int k_1016 = 0; k_1016 < arr_1016.length; k_1016++) {
        sb_1016.append(arr_1016[k_1016]);
        if (k_1016 < arr_1016.length - 1) {
            sb_1016.append(arr_1016[k_1016]);
            if (k_1016 < arr_1016.length - 1) {
                sb_1016.append(", ");
            }
        }
    }
    return sb_1016.toString();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        MergeSortGUI_2511531016 gui_1016 = new MergeSortGUI_2511531016();
        gui_1016.setVisible(true);
    });
}
}

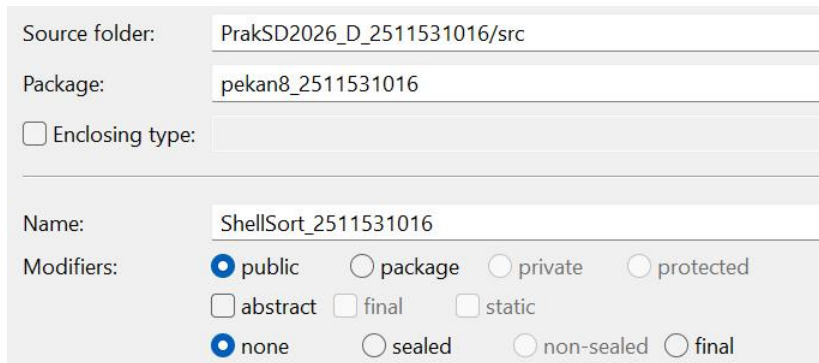
```

Kode Program 2.5 (Kode Program MergeSortGUI)

2.3 Langkah Kerja

2.3.1 ShellSort_2511531016

1. Membuat kelas baru bernama *ShellSort_2511531016* dalam *package* *pekan8_2511531016*.



Gambar 2.1 (Langkah 1 ShellSort)

2. Program diawali dengan deklarasi *package* dan pembuatan *class*. Selanjutnya buat method *shellSort_1016()* yang digunakan untuk melakukan proses pengurutan data menggunakan algoritma Shell Sort. Variabel *n_1016* digunakan untuk menyimpan jumlah elemen array, sedangkan *gap_1016* digunakan sebagai jarak antar elemen yang akan dibandingkan dan diurutkan. Nilai awal *gap* ditentukan sebesar setengah dari jumlah elemen array.

```
package pekan8_2511531016;

public class ShellSort_2511531016 {
    public static void shellSort_1016(int[] A_1016) {
        int n_1016 = A_1016.length;
        int gap_1016 = n_1016 / 2;
        while (gap_1016 > 0) {
```

Gambar 2.2 (Langkah 2 ShellSort)

3. Kemudian, buat perulangan untuk membandingkan dan menggeser elemen berdasarkan nilai *gap_1016*. Nilai yang sedang diproses disimpan ke dalam

variabel temp_1016, kemudian ditempatkan pada posisi yang sesuai. Setelah satu proses selesai, nilai gap dibagi dua hingga mencapai nol.

```
for (int i_1016 = gap_1016; i_1016 < n_1016; i_1016++) {  
    int temp_1016 = A_1016[i_1016];  
    int j_1016 = i_1016;  
    while (j_1016 >= gap_1016 && A_1016[j_1016 - gap_1016] > temp_1016)  
    {  
        A_1016[j_1016] = A_1016[j_1016 - gap_1016];  
        j_1016 = j_1016 - gap_1016;  
    }  
    A_1016[j_1016] = temp_1016;  
}  
gap_1016 = gap_1016 / 2;
```

Gambar 2.3 (Langkah 3 ShellSort)

4. Selanjutnya buat method main() sebagai program utama. Pada bagian ini deklarasikan array yang akan diurutkan, kemudian data ditampilkan sebelum proses pengurutan dan method shellSort_1016() dipanggil untuk mengurutkan data.

```
public static void main(String[] args) {  
    int[] data_1016 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};  
    System.out.print("Sebelum: ");  
    printArray_1016(data_1016);  
    shellSort_1016(data_1016);  
    System.out.print("Sesudah (Shell Sort): ");  
    printArray_1016(data_1016);  
}
```

Gambar 2.4 (Langkah 4 ShellSort)

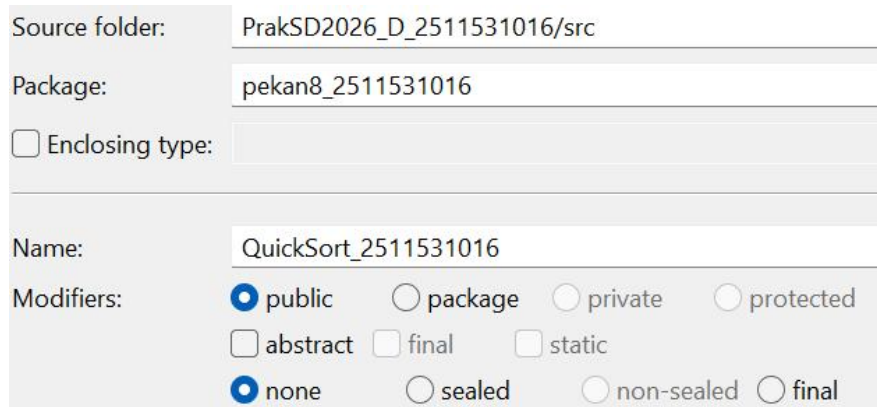
5. Terakhir, buat method printArray_1016() yang digunakan untuk menampilkan isi array ke layar. Method ini dipanggil untuk menampilkan data sebelum dan sesudah proses pengurutan dilakukan.

```
public static void printArray_1016(int[] arr_1016) {  
    for (int i_1016 : arr_1016) System.out.print(i_1016 + " ");  
    System.out.println();  
}
```

Gambar 2.5 (Langkah 5 ShellSort)

2.3.2 QuickSort_2511531016

1. Membuat kelas baru bernama *QuickSort_2511531016* dalam *package pekan8_2511531016*.



Source folder:	PrakSD2026_D_2511531016/src
Package:	pekan8_2511531016
☐ Enclosing type:	
Name:	QuickSort_2511531016
Modifiers:	☒ public ☐ package ☐ private ☐ protected ☐ abstract ☐ final ☐ static ☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.6 (Langkah 1 QuickSort)

2. Deklarasikan *package* dan pembuatan *class*. Lalu, buat method *selectionSort_1016()* yang digunakan untuk menukar posisi dua elemen dalam array selama proses pengurutan berlangsung.

```
package pekan8_2511531016;  
  
public class QuickSort_2511531016 {  
    static void swap_1016(int[] arr_1016, int i_1016, int j_1016) {  
        int temp_1016 = arr_1016[i_1016];  
        arr_1016[i_1016] = arr_1016[j_1016];  
        arr_1016[j_1016] = temp_1016;  
    }  
}
```

Gambar 2.7 (Langkah 2 QuickSort)

3. Kemudian, buat method *medianOfThree_1016()* yang digunakan untuk menentukan pivot dengan metode Median-of-Three. Pada bagian ini juga dihitung indeks tengah array yang akan digunakan dalam proses pemilihan pivot.

```
// Metode tambahan untuk mengatur pivot menggunakan Median-of-Three  
static void medianOfThree_1016(int[] arr_1016, int low_1016, int  
high_1016) {
```

```
int mid_1016 = low_1016 + (high_1016 - low_1016) / 2;
```

Gambar 2.8 (Langkah 3 QuickSort)

4. Lakukan perbandingan antara elemen pertama, tengah, dan terakhir. Ketiga elemen tersebut diurutkan sehingga nilai median dapat digunakan sebagai pivot untuk proses Quick Sort.

```
// Urutkan elemen low, mid, dan high
if (arr_1016[low_1016] > arr_1016[mid_1016]) {
    swap_1016(arr_1016, low_1016, mid_1016);
}
if (arr_1016[low_1016] > arr_1016[high_1016]) {
    swap_1016(arr_1016, low_1016, high_1016);
}
if (arr_1016[mid_1016] > arr_1016[high_1016]) {
    swap_1016(arr_1016, mid_1016, high_1016);
}
swap_1016(arr_1016, mid_1016, high_1016);
```

Gambar 2.9 (Langkah 4 QuickSort)

5. Buat method `partition_1016()` yang berfungsi untuk membagi array berdasarkan nilai pivot. Pivot ditentukan menggunakan method `medianOfThree_1016()` sehingga proses pengurutan menjadi lebih efektif.

```
static int partition_1016(int[] arr_1016, int low_1016, int
high_1016) {
    // Panggil fungsi medianOfThree sebelum menentukan pivot
    medianOfThree_1016 (arr_1016, low_1016, high_1016);
    int pivot_1016 = arr_1016[high_1016]; // Sekarang arr[high] sudah
    berisi nilai median
    int i_1016 = (low_1016 - 1);
```

Gambar 2.10 (Langkah 5 QuickSort)

6. Selanjutnya, lakukan perulangan untuk membandingkan setiap elemen dengan pivot. Jika elemen lebih kecil dari pivot, maka elemen tersebut ditukar posisinya sehingga berada di bagian kiri pivot.

```

for (int j_1016 = low_1016; j_1016 <= high_1016 - 1; j_1016++) {
// Jika elemen saat ini lebih kecil dari atau sama dengan pivot
if (arr_1016[j_1016] < pivot_1016) {
// Increment indeks elemen yang lebih kecil
i_1016++;
swap_1016(arr_1016, i_1016, j_1016);
}
}
swap_1016(arr_1016, i_1016 + 1, high_1016);
return (i_1016 + 1);

```

Gambar 2.11 (Langkah 6 QuickSort)

7. Buat method quickSort_1016() yang menggunakan teknik rekursif. Array akan dibagi menjadi dua bagian berdasarkan posisi pivot, kemudian masing-masing bagian diurutkan kembali hingga seluruh data terurut.

```

static void quickSort_1016(int[] arr_1016, int low_1016, int
high_1016) {
if (low_1016 < high_1016) {
int pi_1016 = partition_1016(arr_1016, low_1016, high_1016);
quickSort_1016(arr_1016, low_1016, pi_1016 - 1);
quickSort_1016(arr_1016, pi_1016 + 1, high_1016);
}
}

```

Gambar 2.12 (Langkah 7 QuickSort)

8. Buat method printArr_1016() yang digunakan untuk menampilkan seluruh isi array ke layar. Method ini digunakan untuk melihat data sebelum dan sesudah proses pengurutan.

```

public static void printArr_1016(int[] arr_1016) {
for (int i_1016 = 0; i_1016 < arr_1016.length; i_1016++) {
System.out.print(arr_1016[i_1016] + " ");
}
System.out.println();
}

```

Gambar 2.13 (Langkah 7 QuickSort)

9. Terakhir, buat method main() sebagai program utama. Pada bagian ini deklarasi data yang akan diurutkan, kemudian data ditampilkan sebelum

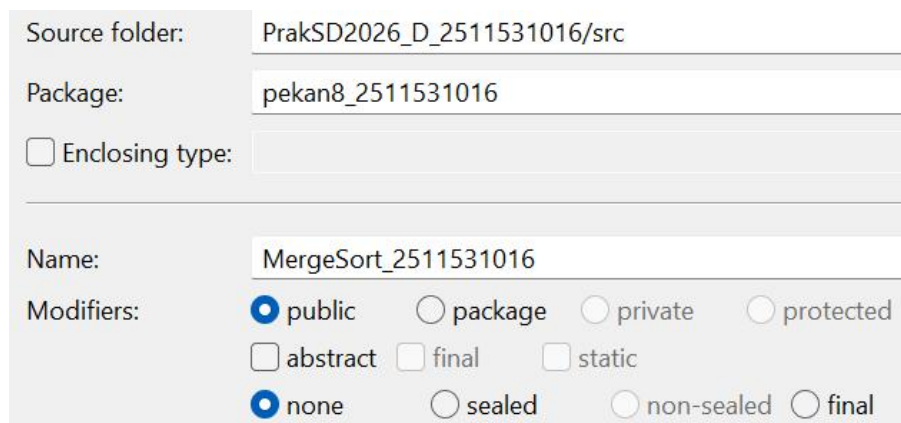
pengurutan, diproses menggunakan method `quickSort_1016()`, dan hasil akhirnya ditampilkan setelah proses pengurutan selesai.

```
public static void main(String[] args) {
    int[] arr_1016 = {10, 7, 8, 9, 1, 5};
    int N_1016 = arr_1016.length;
    System.out.print("data sebelum diurutkan: ");
    printArr_1016(arr_1016);
    quickSort_1016(arr_1016, 0, N_1016 - 1);
    System.out.print("data terurut quicksort: ");
    printArr_1016(arr_1016);
}
```

Gambar 2.14 (Langkah 7 QuickSort)

2.3.3 MergeSort_2511531016

1. Membuat kelas baru bernama *MergeSort_2511531016* dalam package *pekan8_2511531016*.



Source folder:	PrakSD2026_D_2511531016/src
Package:	pekan8_2511531016
☐ Enclosing type:	
Name:	MergeSort_2511531016
Modifiers:	☒ public ☐ package ☐ private ☐ protected ☐ abstract ☐ final ☐ static ☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.15 (Langkah 1 MergeSort)

2. Program dimulai dengan deklarasi package dan pembuatan class. Lalu, buat method `merge_1016()` yang digunakan untuk menggabungkan dua subarray yang telah terurut. Pada bagian ini juga ditentukan ukuran masing-masing subarray dan dibuat array sementara untuk menampung data.

```
package pekan8_2511531016;
```

```

public class MergeSort_2511531016 {
void merge_1016(int arr_1016[], int l_1016, int m_1016, int r_1016)
{
// Find sizes of two subarrays to be merged
int n1_1016 = m_1016 - l_1016 + 1;
int n2_1016 = r_1016 - m_1016;
/* Create temp arrays */
int L_1016[] = new int[n1_1016];
int R_1016[] = new int[n2_1016];

```

Gambar 2.16 (Langkah 2 MergeSort)

3. Kemudian, data dari array utama disalin ke dalam array sementara L_1016 dan R_1016. Setelah itu dibuat variabel indeks i_1016 dan j_1016 yang digunakan untuk mengakses elemen pada kedua subarray.

```

/* Copy data to temp arrays */
for (int i_1016 = 0; i_1016 < n1_1016; ++i_1016)
L_1016[i_1016] = arr_1016[l_1016 + i_1016];

for (int j_1016 = 0; j_1016 < n2_1016; ++j_1016)
R_1016[j_1016] = arr_1016[m_1016 + 1 + j_1016];

int i_1016 = 0, j_1016 = 0;

```

Gambar 2.17 (Langkah 3 MergeSort)

4. Lakukan proses penggabungan dua subarray dengan membandingkan elemen dari array L_1016 dan R_1016. Nilai yang lebih kecil akan dimasukkan ke array utama hingga salah satu subarray habis.

```

// Initial index of merged subarray array
int k_1016 = l_1016;

while (i_1016 < n1_1016 && j_1016 < n2_1016) {
if (L_1016[i_1016] <= R_1016[j_1016]) {
arr_1016[k_1016] = L_1016[i_1016];
i_1016++;
} else {
arr_1016[k_1016] = R_1016[j_1016];
j_1016++;
}
k_1016++;
}

```


Gambar 2.18 (Langkah 4 MergeSort)

5. Setelah proses perbandingan selesai, elemen yang masih tersisa pada salah satu subarray akan disalin ke array utama hingga seluruh data berhasil digabungkan.

```
/* Copy remaining elements of L[] if any */
while (i_1016 < n1_1016) {
    arr_1016[k_1016] = L_1016[i_1016];
    i_1016++;
    k_1016++;
}

/* Copy remaining elements of R[] if any */
while (j_1016 < n2_1016) {
    arr_1016[k_1016] = R_1016[j_1016];
    j_1016++;
    k_1016++;
}
```

Gambar 2.19 (Langkah 5 MergeSort)

6. Selanjutnya, buat method `sort_1016()` yang berfungsi untuk membagi array menjadi dua bagian secara rekursif. Setelah kedua bagian selesai diurutkan, method `merge_1016()` dipanggil untuk menggabungkannya kembali.

```
void sort_1016(int arr_1016[], int l_1016, int r_1016) {
    if (l_1016 < r_1016) {
        // Find the middle point
        int m_1016 = (l_1016 + r_1016) / 2;
        // Sort first and second halves
        sort_1016(arr_1016, l_1016, m_1016);
        sort_1016(arr_1016, m_1016 + 1, r_1016);
        // Merge the sorted halves
        merge_1016(arr_1016, l_1016, m_1016, r_1016);
    }
}
```

Gambar 2.20 (Langkah 6 MergeSort)

7. Buat method `printArray_1016()` yang digunakan untuk menampilkan isi array sebelum dan sesudah proses pengurutan dilakukan.

```

/* A utility function to print array of size n */
static void printArray_1016(int arr_1016[]) {
    int n_1016 = arr_1016.length;
    for (int i_1016 = 0; i_1016 < n_1016; ++i_1016)
        System.out.print(arr_1016[i_1016] + " ");
    System.out.println();
}

```

Gambar 2.21 (Langkah 7 MergeSort)

8. Terakhir, buat method `main()` sebagai program utama. Pada bagian ini deklarasikan data yang akan diurutkan, kemudian method `sort_1016()` dipanggil untuk melakukan pengurutan menggunakan algoritma Merge Sort dan hasilnya ditampilkan ke layar.

```

public static void main(String args[]) {
    int arr_1016[] = {12, 11, 13, 5, 6, 7};
    System.out.println("Sebelum terurut");
    printArray_1016(arr_1016);

    MergeSort_2511531016 ob_1016 = new MergeSort_2511531016();
    ob_1016.sort_1016(arr_1016, 0, arr_1016.length - 1);

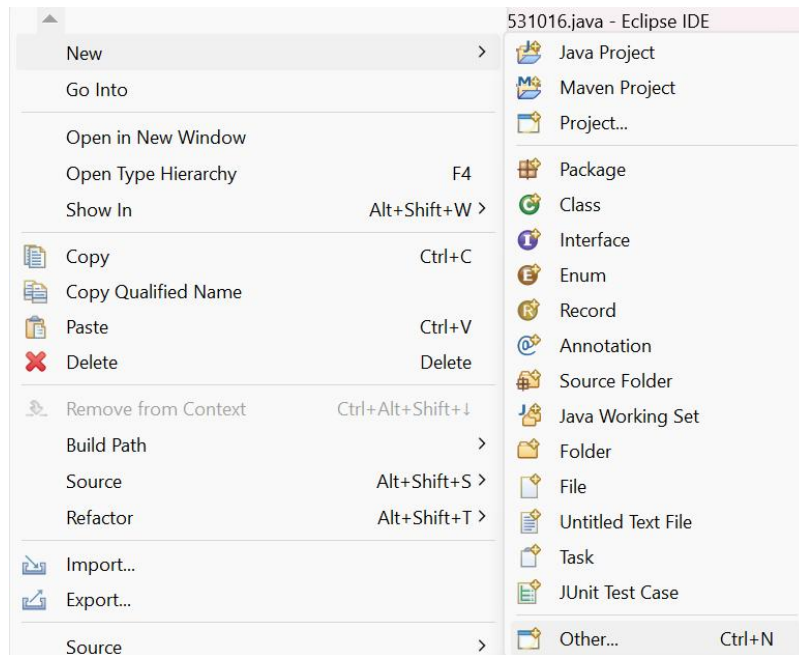
    System.out.println("\nSesudah Terurut menggunakan merge Sort");
    printArray_1016(arr_1016);
}

```

Gambar 2.22 (Langkah 7 MergeSort)

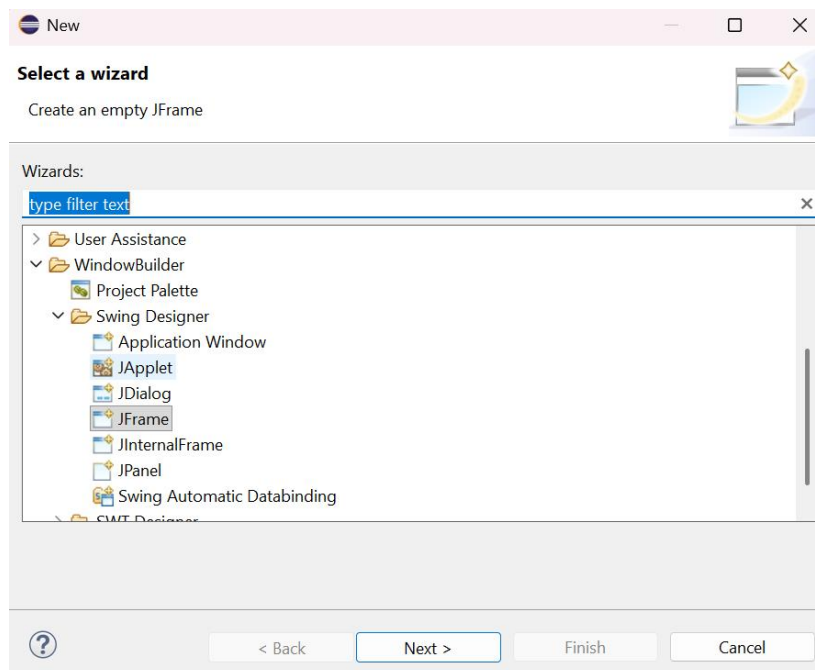
2.3.4 BubbleSortGUI_2511531016

1. Untuk membuat class GUI, klik kanan pada package `pekan8_2511531016`, lalu pilih “New”, pilih “Other”.



Gambar 2.23 (Langkah 1 BubbleSortGUI)

2. Selanjutnya, klik WindowBuilder dan pilih JFrame. Klik Next.



Gambar 2.24 (Langkah 2 BubbleSortGUI)

3. Kemudian, isikan nama class yaitu BubbleSortGUI_2511531016. Klik Finish.

Source	PrakSD2026_D_2511531016/src
Package:	pekan8_2511531016
Name:	BubbleSortGUI_2511531016
Superclass:	javax.swing.JFrame
☒ Use advanced template for generate JFrame	

Gambar 2.25 (Langkah 3 BubbleSortGUI)

4. Program diawali dengan deklarasi package serta import beberapa library Java Swing dan AWT yang digunakan untuk membuat tampilan GUI. Library tersebut dipakai untuk membuat frame, panel, tombol, text field, label, area teks, hingga pengaturan warna dan font pada aplikasi visualisasi Bubble Sort.

```
package pekan8_2511531016;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextArea;
import javax.swing.border.EmptyBorder;
```

Gambar 2.26 (Langkah 4 BubbleSortGUI)

5. Kemudian, deklarasikan class `BubbleSortGUI_2511531016` yang merupakan turunan dari `JFrame`. Pada bagian ini juga buat beberapa variabel seperti array, label array, tombol, text field, panel, dan text area yang digunakan untuk menampilkan proses pengurutan secara visual pada GUI.

```
public class BubbleSortGUI_2511531016 extends JFrame {
    private static final long serialVersionUID = 1L;
    private JPanel contentPane_1016;
    private int[] array_1016;
    private JLabel[] labelArray_1016;
    private JButton stepButton_1016, resetButton_1016, setButton_1016;
    private JTextField inputField_1016;
    private JPanel panelArray_1016;
    private JTextArea stepArea_1016;
    private int i_1016 = 1, j_1016;
    private boolean sorting_1016 = false;
    private int stepCount_1016 = 1;
```

Gambar 2.27 (Langkah 5 BubbleSortGUI)

6. Buat constructor `BubbleSortGUI_2511531016()` untuk membangun tampilan utama program. Pada bagian ini diatur ukuran frame, judul aplikasi, posisi tampilan, layout, serta panel utama yang akan menampung seluruh komponen GUI.

```
/**
 * Create the frame.
 */
public BubbleSortGUI_2511531016() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 450, 300);
    contentPane_1016 = new JPanel();
    contentPane_1016.setBorder(new EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane_1016);

    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());
```

Gambar 2.28 (Langkah 6 BubbleSortGUI)

7. Lalu, buat panel input menggunakan JPanel dengan layout FlowLayout. Pada panel ini terdapat JTextField untuk memasukkan data array dan tombol Set Array yang digunakan untuk memproses input angka dari pengguna.

```
// Panel input
JPanel inputPanel_1016 = new JPanel(new FlowLayout());
inputField_1016 = new JTextField(30);
setButton_1016 = new JButton("Set Array");
inputPanel_1016.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
inputPanel_1016.add(inputField_1016);
inputPanel_1016.add(setButton_1016);
```

Gambar 2.29 (Langkah 7 BubbleSortGUI)

8. Setelah itu dibuat panel visual array bernama panelArray_1016. Panel ini digunakan untuk menampilkan isi array dalam bentuk label sehingga proses pengurutan dapat dilihat secara visual langkah demi langkah.

```
// Panel array visual
panelArray_1016 = new JPanel();
panelArray_1016.setLayout(new FlowLayout());
```

Gambar 2.30 (Langkah 8 BubbleSortGUI)

9. Buat panel kontrol yang berisi tombol Langkah Selanjutnya dan Reset. Tombol langkah digunakan untuk menjalankan proses sorting satu per satu, sedangkan tombol reset dipakai untuk mengembalikan tampilan program ke kondisi awal.

```
// Panel kontrol
JPanel controlPanel_1016 = new JPanel();
stepButton_1016 = new JButton("Langkah Selanjutnya");
resetButton_1016 = new JButton("Reset");
stepButton_1016.setEnabled(false);
controlPanel_1016.add(stepButton_1016);
controlPanel_1016.add(resetButton_1016);
```

Gambar 2.31 (Langkah 9 BubbleSortGUI)

10. Selanjutnya, buat JTextArea bernama stepArea_1016 yang digunakan untuk menampilkan log atau penjelasan setiap langkah proses sorting. Area teks ini dimasukkan ke dalam JScrollPane agar pengguna dapat melihat seluruh riwayat langkah meskipun teks sudah panjang.

```
// Area teks untuk log langkah-langkah
stepArea_1016 = new JTextArea(8, 60);
stepArea_1016.setEditable(false);
stepArea_1016.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane_1016 = new JScrollPane(stepArea_1016);
```

Gambar 2.32 (Langkah 10 BubbleSortGUI)

11. Setelah semua komponen dibuat, tambahkan panel-panel tersebut ke dalam frame menggunakan BorderLayout. Panel input ditempatkan di bagian atas, panel array di tengah, panel kontrol di bawah, dan area log di sisi kanan tampilan.

```
// Tambahkan panel ke frame
add(inputPanel_1016, BorderLayout.NORTH);
add(panelArray_1016, BorderLayout.CENTER);
add(controlPanel_1016, BorderLayout.SOUTH);
add(scrollPane_1016, BorderLayout.EAST);
```

Gambar 2.33 (Langkah 11 BubbleSortGUI)

12. Tambahkan event listener pada setiap tombol. Tombol Set Array digunakan untuk membaca input array, tombol Langkah Selanjutnya dipakai untuk menjalankan satu langkah sorting, sedangkan tombol Reset digunakan untuk menghapus data dan mengulang program dari awal.

```
// Event Set Array
setButton_1016.addActionListener(e -> setArrayFromInput_1016());

// Event Langkah Selanjutnya
stepButton_1016.addActionListener(e -> performStep_1016());

// Event Reset
resetButton_1016.addActionListener(e -> reset_1016());
```

Gambar 2.34 (Langkah 12 BubbleSortGUI)

13. Pada method `setArrayFromInput_1016()`, program membaca input angka dari pengguna yang dipisahkan dengan tanda koma. Data input tersebut dipecah menggunakan `split(",")`, lalu dikonversi menjadi tipe integer dan disimpan ke dalam array.

```
private void setArrayFromInput_1016() {
    String text_1016 = inputField_1016.getText().trim();
    if (text_1016.isEmpty()) return;
    String[] parts_1016 = text_1016.split(",");
    array_1016 = new int[parts_1016.length];
    try {
        for (int k_1016 = 0; k_1016 < parts_1016.length; k_1016++) {
            array_1016[k_1016] = Integer.parseInt(parts_1016[k_1016].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka " + "yang dipisahkan koma!", "Error", JOptionPane.ERROR_MESSAGE);
    }
    return;
}
```

Gambar 2.35 (Langkah 13 BubbleSortGUI)

14. Lakukan inisialisasi variabel yang digunakan selama proses Bubble Sort berlangsung. Nilai `i_1016`, `j_1016`, dan `stepCount_1016` diatur ke nilai awal, kemudian tombol langkah diaktifkan dan area log dikosongkan. Selanjutnya dibuat label untuk setiap elemen array agar data dapat ditampilkan secara visual pada GUI.

```
i_1016 = 0;
j_1016 = 0;
stepCount_1016 = 1;
sorting_1016 = true;
stepButton_1016.setEnabled(true);
stepArea_1016.setText("");
panelArray_1016.removeAll();
labelArray_1016 = new JLabel[array_1016.length];
for (int k_1016 = 0; k_1016 < array_1016.length; k_1016++) {
    labelArray_1016[k_1016] = new
    JLabel(String.valueOf(array_1016[k_1016]));
    labelArray_1016[k_1016].setFont(new Font("Arial", Font.BOLD, 24));
    labelArray_1016[k_1016].setOpaque(true);
    labelArray_1016[k_1016].setBackground(Color.WHITE);
}
```



```

labelArray_1016[k_1016].setBorder(BorderFactory.createLineBorder(Color.BLACK));
labelArray_1016[k_1016].setPreferredSize(new Dimension(50, 50));
labelArray_1016[k_1016].setHorizontalAlignment(SwingConstants.CENTER);
;
panelArray_1016.add(labelArray_1016[k_1016]);
}
panelArray_1016.revalidate();
panelArray_1016.repaint();

```

Gambar 2.36 (Langkah 14 BubbleSortGUI)

15. Buat method performStep_1016() yang digunakan untuk menjalankan satu langkah proses Bubble Sort setiap kali tombol ditekan. Pada bagian ini lakukan pengecekan apakah proses sorting masih berlangsung, kemudian dua elemen yang sedang dibandingkan akan diberi warna sebagai penanda pada tampilan GUI.

```

private void performStep_1016() {
if (!sorting_1016 || i_1016 >= array_1016.length - 1) {
sorting_1016 = false;
stepButton_1016.setEnabled(false);
JOptionPane.showMessageDialog(this, "Sorting selesai!");
return;
}
resetHighlights();
StringBuilder stepLog = new StringBuilder();
labelArray_1016[j_1016].setBackground(Color.CYAN);
labelArray_1016[j_1016 + 1].setBackground(Color.CYAN);
if (array_1016[j_1016] > array_1016[j_1016 + 1]) {

```

Gambar 2.37 (Langkah 15 BubbleSortGUI)

16. Jika elemen sebelah kiri lebih besar daripada elemen sebelah kanan, maka lakukan proses pertukaran data (swap). Setelah itu informasi pertukaran dicatat ke dalam area log sehingga pengguna dapat melihat setiap langkah pengurutan yang terjadi.

```

// swap
int temp_1016 = array_1016[j_1016];
array_1016[j_1016] = array_1016[j_1016 + 1];
array_1016[j_1016 + 1] = temp_1016;
labelArray_1016[j_1016].setBackground(Color.RED);

```

```

labelArray_1016[j_1016 + 1].setBackground(Color.RED);

stepLog.append("Langkah ").append(stepCount_1016)
.append(": Menukar elemen ke-")
.append(j_1016).append(" [")
.append(array_1016[j_1016 + 1]).append("] dengan ke-")
.append(j_1016 + 1).append(" [")
.append(array_1016[j_1016]).append("]\n");
} else {
stepLog.append("Langkah ").append(stepCount_1016)
.append(": Tidak ada pertukaran antara ke-")
.append(j_1016).append(" dan ke-")
.append(j_1016 + 1).append("\n");
}

stepLog.append("Hasil: ")
.append(arrayToString_1016(array_1016))
.append("\n\n");
stepArea_1016.append(stepLog.toString());
updateLabels_1016();
j_1016++;

if (j_1016 >= array_1016.length - i_1016 - 1) {
j_1016 = 0;
i_1016++;
}
stepCount_1016++;

if (i_1016 >= array_1016.length - 1) {
sorting_1016 = false;
stepButton_1016.setEnabled(false);
JOptionPane.showMessageDialog(this, "Sorting selesai!");
}

```

Gambar 2.38 (Langkah 16 BubbleSortGUI)

17. Method updateLabels_1016() digunakan untuk memperbarui isi setiap label sesuai data terbaru dalam array. Dengan cara ini tampilan GUI akan selalu menyesuaikan hasil sorting terbaru setelah setiap langkah dijalankan.

```

private void updateLabels_1016() {
for (int k_1016 = 0; k_1016 < array_1016.length; k_1016++) {
labelArray_1016[k_1016].setText(String.valueOf(array_1016[k_1016]));
}
}

```

Gambar 2.39 (Langkah 17 BubbleSortGUI)

18. Buat method `resetHighlights()` yang berfungsi untuk mengembalikan warna seluruh label menjadi putih sebelum proses perbandingan berikutnya dilakukan. Hal ini membantu pengguna melihat dengan jelas elemen yang sedang diproses pada setiap langkah sorting.

```
private void resetHighlights() {
    for(JLabel label_1016 : labelArray_1016) {
        label_1016.setBackground(Color.WHITE);
    }
}
```

Gambar 2.40 (Langkah 18 BubbleSortGUI)

19. Pada method `reset_1016()`, hapus seluruh input, bersihkan panel array dan area log, lalu kembalikan variabel sorting ke kondisi awal. Tombol langkah juga dinonaktifkan kembali sampai pengguna memasukkan data baru.

```
private void reset_1016() {
    inputField_1016.setText("");
    panelArray_1016.removeAll();
    panelArray_1016.revalidate();
    panelArray_1016.repaint();
    stepArea_1016.setText("");
    stepButton_1016.setEnabled(false);
    sorting_1016 = false;
    i_1016 = 0;
    j_1016 = 0;
    stepCount_1016 = 1;
}
```

Gambar 2.41 (Langkah 19 BubbleSortGUI)

20. Method `arrayToString_1016()` digunakan untuk mengubah isi array menjadi bentuk string. Setiap elemen array digabungkan dengan tanda koma sehingga hasil array lebih mudah ditampilkan pada area log langkah-langkah sorting.

```
private String arrayToString_1016(int[] arr_1016) {
    StringBuilder sb_1016 = new StringBuilder();
    for (int k_1016 = 0; k_1016 < arr_1016.length; k_1016++) {
        sb_1016.append(arr_1016[k_1016]);
        if (k_1016 < arr_1016.length - 1) sb_1016.append(", ");
    }
}
```

```
return sb_1016.toString();
```

Gambar 2.42 (Langkah 20 BubbleSortGUI)

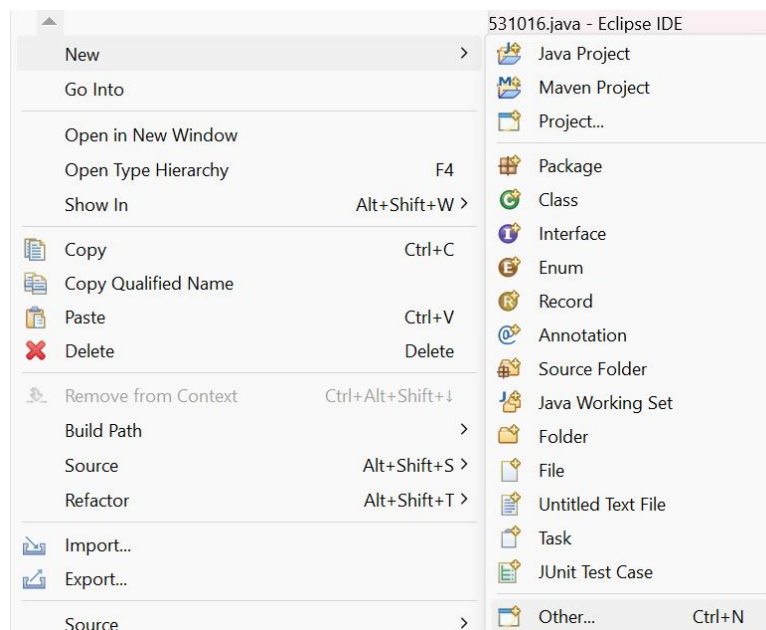
21. Terakhir, pada method `main()`, jalankan GUI menggunakan `SwingUtilities.invokeLater()`. Setelah objek `BubbleSortGUI_2511531016` dibuat, tampilan aplikasi akan muncul dan siap digunakan oleh pengguna untuk melihat proses Bubble Sort secara visual.

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        BubbleSortGUI_2511531016 gui_1016 = new BubbleSortGUI_2511531016();
        gui_1016.setVisible(true);
    });
}
```

Gambar 2.43 (Langkah 21 BubbleSortGUI)

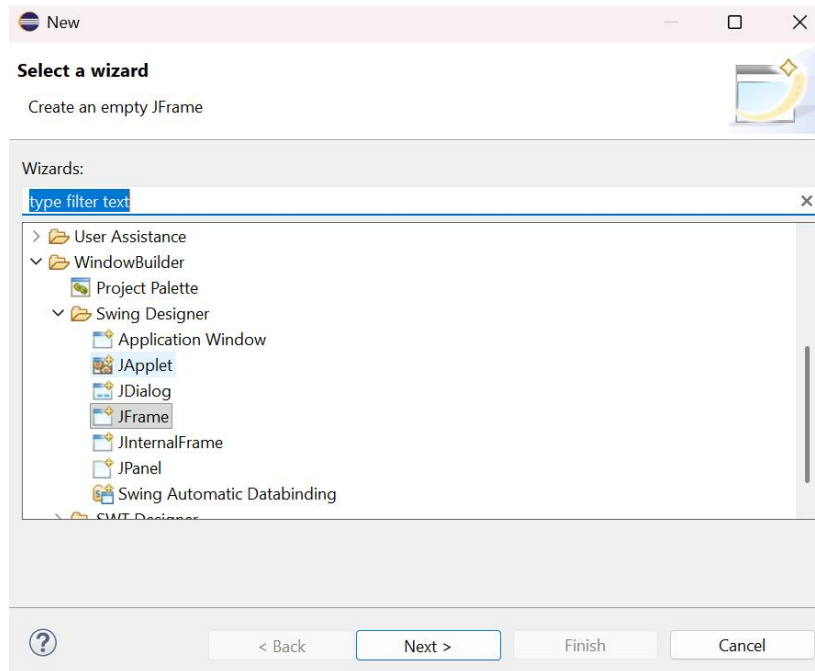
2.3.5 MergeSortGUI_2511531016

1. Untuk membuat class GUI, klik kanan pada package `pekan8_2511531016`, lalu pilih “New”, pilih “Other”.



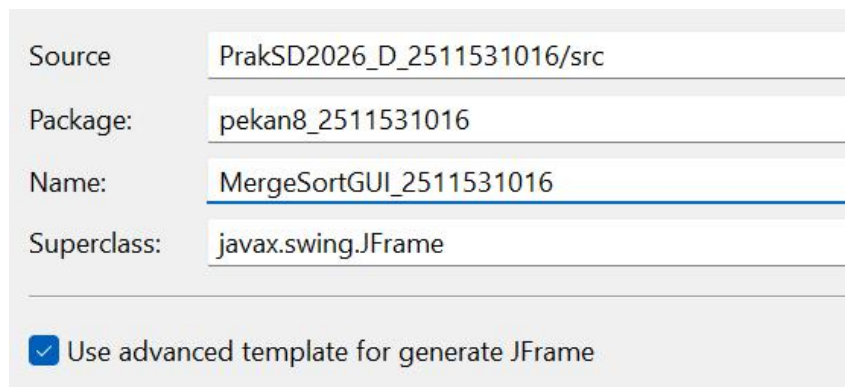
Gambar 2.44 (Langkah 1 MergeSortGUI)

2. Selanjutnya, klik WindowBuilder dan pilih JFrame. Klik Next.



Gambar 2.45 (Langkah 2 MergeSortGUI)

3. Kemudian, isikan nama class yaitu MergeSortGUI_2511531016. Klik Finish.



Gambar 2.46 (Langkah 3 MergeSortGUI)

4. Program diawali dengan deklarasi package serta import beberapa library Java Swing dan AWT yang digunakan untuk membuat tampilan GUI. Library

tersebut dipakai untuk membuat frame, panel, tombol, text field, label, area teks, hingga pengaturan warna dan font pada aplikasi visualisasi Merge Sort.

```
package pekan8_2511531016;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;
import java.util.Queue;
import java.util.LinkedList;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextArea;
import javax.swing.border.EmptyBorder;
```

Gambar 2.47 (Langkah 4 MergeSortGUI)

5. Kemudian, deklarasikan class MergeSortGUI_2511531016 yang merupakan turunan dari JFrame. Pada bagian ini dideklarasikan berbagai komponen GUI seperti panel, tombol, text field, dan label. Selain itu juga dibuat beberapa variabel yang digunakan untuk mengatur proses Merge Sort, seperti array data, queue untuk menyimpan langkah merge, serta indeks yang diperlukan selama proses pengurutan.

```
public class MergeSortGUI_2511531016 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane_1016;
    private int[] array_1016;
    private JLabel[] labelArray_1016;
    private JButton stepButton_1016, resetButton_1016, setButton_1016;
    private JTextField inputField_1016;
    private JPanel panelArray_1016;
    private JTextArea stepArea_1016;
    private boolean sorting_1016 = false;
```

```

private boolean copying_1016 = false;
private boolean isMerging_1016 = false;
private int stepCount_1016 = 1;
private java.util.Queue<int[]> mergeQueue_1016 = new
java.util.LinkedList<>();
private int left_1016, mid_1016, right_1016;
private int[] temp_1016;
private int i_1016, j_1016, k_1016;

```

Gambar 2.48 (Langkah 5 MergeSortGUI)

6. Buat constructor MergeSortGUI_2511531016() untuk membangun tampilan utama program. Pada bagian ini diatur ukuran frame, judul aplikasi, posisi tampilan, layout, serta panel utama yang akan menampung seluruh komponen GUI.

```

/**
 * Create the frame.
 */
public MergeSortGUI_2511531016() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 450, 300);
    contentPane_1016 = new JPanel();
    contentPane_1016.setBorder(new EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane_1016);

    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

```

Gambar 2.49 (Langkah 6 MergeSortGUI)

7. Lalu, buat panel input menggunakan JPanel dengan layout FlowLayout. Pada panel ini terdapat JTextField untuk memasukkan data array dan tombol Set Array yang digunakan untuk memproses input angka dari pengguna.

```

// Panel input
JPanel inputPanel_1016 = new JPanel(new FlowLayout());
inputField_1016 = new JTextField(30);
setButton_1016 = new JButton("Set Array");
inputPanel_1016.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));

```

```
inputPanel_1016.add(inputField_1016);
inputPanel_1016.add(setButton_1016);
```

Gambar 2.50 (Langkah 7 MergeSortGUI)

8. Setelah itu dibuat panel visual array bernama panelArray_1016. Panel ini digunakan untuk menampilkan isi array dalam bentuk label sehingga proses pengurutan dapat dilihat secara visual langkah demi langkah.

```
// Panel array visual
panelArray_1016 = new JPanel();
panelArray_1016.setLayout(new FlowLayout());
```

Gambar 2.51 (Langkah 8 MergeSortGUI)

9. Buat panel kontrol yang berisi tombol Langkah Selanjutnya dan Reset. Tombol langkah digunakan untuk menjalankan proses sorting satu per satu, sedangkan tombol reset dipakai untuk mengembalikan tampilan program ke kondisi awal.

```
// Panel kontrol
JPanel controlPanel_1016 = new JPanel();
stepButton_1016 = new JButton("Langkah Selanjutnya");
resetButton_1016 = new JButton("Reset");
stepButton_1016.setEnabled(false);
controlPanel_1016.add(stepButton_1016);
controlPanel_1016.add(resetButton_1016);
```

Gambar 2.52 (Langkah 9 MergeSortGUI)

10. Selanjutnya, buat JTextArea bernama stepArea_1016 yang digunakan untuk menampilkan log atau penjelasan setiap langkah proses sorting. Area teks ini dimasukkan ke dalam JScrollPane agar pengguna dapat melihat seluruh riwayat langkah meskipun teks sudah panjang.

```
// Area teks untuk log langkah-langkah
stepArea_1016 = new JTextArea(8, 60);
stepArea_1016.setEditable(false);
```



```
stepArea_1016.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane_1016 = new JScrollPane(stepArea_1016);
```

Gambar 2.53 (Langkah 10 MergeSortGUI)

11. Setelah semua komponen dibuat, tambahkan panel-panel tersebut ke dalam frame menggunakan BorderLayout. Panel input ditempatkan di bagian atas, panel array di tengah, panel kontrol di bawah, dan area log di sisi kanan tampilan.

```
// Tambahkan panel ke frame
add(inputPanel_1016, BorderLayout.NORTH);
add(panelArray_1016, BorderLayout.CENTER);
add(controlPanel_1016, BorderLayout.SOUTH);
add(scrollPane_1016, BorderLayout.EAST);
```

Gambar 2.54 (Langkah 11 MergeSortGUI)

12. Tambahkan event listener pada setiap tombol. Tombol Set Array digunakan untuk membaca input array, tombol Langkah Selanjutnya dipakai untuk menjalankan satu langkah sorting, sedangkan tombol Reset digunakan untuk menghapus data dan mengulang program dari awal.

```
// Event Set Array
setButton_1016.addActionListener(e -> setArrayFromInput_1016());

// Event Langkah Selanjutnya
stepButton_1016.addActionListener(e -> performStep_1016());

// Event Reset
resetButton_1016.addActionListener(e -> reset_1016());
```

Gambar 2.55 (Langkah 12 MergeSortGUI)

13. Pada method setArrayFromInput_1016(), program membaca input angka dari pengguna yang dipisahkan dengan tanda koma. Data input tersebut dipecah menggunakan split(","), lalu dikonversi menjadi tipe integer dan disimpan ke dalam array.

```

private void setArrayFromInput_1016() {
String text = inputField_1016.getText().trim();
if (text.isEmpty()) return;
String[] parts = text.split(",");
array_1016 = new int[parts.length];
try {
for (int i = 0; i < parts.length; i++) {
array_1016[i] = Integer.parseInt(parts[i].trim());
}
} catch (NumberFormatException e) {
JOptionPane.showMessageDialog(this, "Masukkan hanya angka!", "Error",
JOptionPane.ERROR_MESSAGE);
return;
}
}

```

Gambar 2.56 (Langkah 13 MergeSortGUI)

14. Buat tampilan visual array menggunakan komponen JLabel. Setiap elemen array ditampilkan dalam bentuk kotak sehingga proses pengurutan dapat diamati secara langsung pada GUI. Setelah itu queue merge diinisialisasi dan langkah-langkah merge disiapkan

```

labelArray_1016 = new JLabel[array_1016.length];
panelArray_1016.removeAll();

for (int i = 0; i < array_1016.length; i++) {
labelArray_1016[i] = new JLabel(String.valueOf(array_1016[i]));
labelArray_1016[i].setFont(new Font("Arial", Font.BOLD, 24));
labelArray_1016[i].setOpaque(true);
labelArray_1016[i].setBackground(Color.WHITE);
labelArray_1016[i].setBorder(BorderFactory.createLineBorder(Color.BLUE));
labelArray_1016[i].setPreferredSize(new Dimension(50, 50));
labelArray_1016[i].setHorizontalAlignment(SwingConstants.CENTER);
panelArray_1016.add(labelArray_1016[i]);
}
mergeQueue_1016.clear();
generateMergeSteps_1016(0, array_1016.length - 1);
stepButton_1016.setEnabled(true);
stepArea_1016.setText("");
stepCount_1016 = 1;
isMerging_1016 = false;
panelArray_1016.revalidate();
panelArray_1016.repaint();
}

```

Gambar 2.57 (Langkah 14 MergeSortGUI)

15. Buat method `generateMergeSteps_1016()` yang berfungsi untuk membagi array menjadi beberapa bagian secara rekursif. Setiap rentang indeks yang akan digabungkan disimpan ke dalam queue agar dapat diproses satu per satu pada saat tombol langkah ditekan.

```
private void generateMergeSteps_1016(int left_1016, int right_1016)
{
    if (left_1016 >= right_1016) {
        return;
    }
    int mid_1016 = (left_1016 + right_1016) / 2;

    generateMergeSteps_1016(left_1016, mid_1016);
    generateMergeSteps_1016(mid_1016 + 1, right_1016);

    mergeQueue_1016.add(new int[] {
        left_1016, mid_1016, right_1016
    });
}
```

Gambar 2.58 (Langkah 15 MergeSortGUI)

16. Selanjutnya, buat method `performStep_1016()` yang digunakan untuk menjalankan proses Merge Sort langkah demi langkah. Pada bagian ini diambil rentang data dari queue, kemudian ditentukan indeks kiri, tengah, dan kanan yang akan digabungkan.

```
private void performStep_1016() {
    resetHighlights_1016();
    if (!isMerging_1016 && !mergeQueue_1016.isEmpty()) {
        int[] range_1016 = mergeQueue_1016.poll();

        left_1016 = range_1016[0];
        mid_1016 = range_1016[1];
        right_1016 = range_1016[2];
        temp_1016 = new int[right_1016 - left_1016 + 1];

        i_1016 = left_1016;
        j_1016 = mid_1016 + 1;
        k_1016 = 0;

        copying_1016 = false;
        isMerging_1016 = true;
        stepArea_1016.append("Langkah " + stepCount_1016++ + ": Mulai merge dari indeks " + left_1016 + " sampai " + right_1016 + "\n");
        return;
    }
}
```

Gambar 2.59 (Langkah 16 MergeSortGUI)

17. Lakukan proses perbandingan antara elemen pada bagian kiri dan kanan. Elemen yang lebih kecil akan disalin ke array sementara dan setiap langkah dicatat pada area log agar pengguna dapat melihat proses merge yang sedang berlangsung.

```
if (isMerging_1016 && !copying_1016) {
if (i_1016 <= mid_1016 && j_1016 <= right_1016) {
labelArray_1016[i_1016].setBackground(Color.CYAN);
labelArray_1016[j_1016].setBackground(Color.CYAN);
if (array_1016[i_1016] <= array_1016[j_1016]) {
temp_1016[k_1016++] = array_1016[i_1016++];
} else {
temp_1016[k_1016++] = array_1016[j_1016++];
}
stepArea_1016.append(
"Langkah " + stepCount_1016++ + ": Bandingkan dan salin elemen\n"
+ "Hasil: " + arrayToString_1016(array_1016) + "\n\n");
return;
}
```

Gambar 2.60 (Langkah 17 MergeSortGUI)

18. Jika masih terdapat elemen yang belum disalin pada salah satu bagian array, maka elemen tersebut akan dipindahkan ke array sementara hingga seluruh data selesai diproses.

```
else if (i_1016 <= mid_1016) {
temp_1016[k_1016++] = array_1016[i_1016++];
stepArea_1016.append(
"Langkah " + stepCount_1016++ + ": Salin sisa kiri\n" + "Hasil: "
+ arrayToString_1016(array_1016) + "\n\n");
return;
}
else if (j_1016 <= right_1016) {
temp_1016[k_1016++] = array_1016[j_1016++];
stepArea_1016.append(
"Langkah " + stepCount_1016++ + ": Salin sisa kanan\n" + "Hasil: "
+ arrayToString_1016(array_1016) + "\n\n");
return;
}
else {
copying_1016 = true;
k_1016 = 0;
}
```

```
return;
}
```

Gambar 2.61 (Langkah 18 MergeSortGUI)

19. Setelah semua elemen berada pada array sementara, data akan ditempelkan kembali ke array utama. Jika seluruh proses merge telah selesai dan queue kosong, maka program akan menampilkan pesan bahwa proses Merge Sort telah berakhir.

```
if (copying_1016 && k_1016 < temp_1016.length) {
    array_1016[left_1016 + k_1016] = temp_1016[k_1016];
    labelArray_1016[left_1016 +
    k_1016].setText(String.valueOf(temp_1016[k_1016]));
    labelArray_1016[left_1016 + k_1016].setBackground(Color.GREEN);
    k_1016++;
    stepArea_1016.append(
    "Langkah " + stepCount_1016++ + ": Tempelkan ke array utama\n"
    + "Hasil: " + arrayToString_1016(array_1016) + "\n\n");
    return;
}
if (copying_1016 && k_1016 == temp_1016.length) {
    isMerging_1016 = false;
    copying_1016 = false;
}
if (mergeQueue_1016.isEmpty() && !isMerging_1016) {
    stepArea_1016.append("Selesai.\n");
    stepButton_1016.setEnabled(false);
    JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
}
```

Gambar 2.62 (Langkah 19 MergeSortGUI)

20. Buat method resetHighlights() yang berfungsi untuk mengembalikan warna seluruh label menjadi putih sebelum proses perbandingan berikutnya dilakukan. Hal ini membantu pengguna melihat dengan jelas elemen yang sedang diproses pada setiap langkah sorting.

```
private void resetHighlights_1016() {
    if (labelArray_1016 == null) {
        return;
    }
    for (JLabel label_1016 : labelArray_1016) {
        label_1016.setBackground(Color.WHITE);
    }
}
```

Gambar 2.63 (Langkah 20 MergeSortGUI)

21. Pada method `reset_1016()`, hapus seluruh input, bersihkan panel array dan area log, lalu kembalikan variabel sorting ke kondisi awal. Tombol langkah juga dinonaktifkan kembali sampai pengguna memasukkan data baru.

```
private void reset_1016() {
    inputField_1016.setText("");
    panelArray_1016.removeAll();
    panelArray_1016.revalidate();
    panelArray_1016.repaint();
    stepArea_1016.setText("");
    stepButton_1016.setEnabled(false);
    mergeQueue_1016.clear();
    isMerging_1016 = false;
    stepCount_1016 = 1;
}
```

Gambar 2.64 (Langkah 21 MergeSortGUI)

22. Method `arrayToString_1016()` digunakan untuk mengubah isi array menjadi bentuk string. Setiap elemen array digabungkan dengan tanda koma sehingga hasil array lebih mudah ditampilkan pada area log langkah-langkah sorting.

```
private String arrayToString_1016(int[] arr_1016) {
    StringBuilder sb_1016 = new StringBuilder();
    for (int k_1016 = 0; k_1016 < arr_1016.length; k_1016++) {
        sb_1016.append(arr_1016[k_1016]);
        if (k_1016 < arr_1016.length - 1) {
            sb_1016.append(", ");
        }
    }
    return sb_1016.toString();
}
```

Gambar 2.65 (Langkah 22 MergeSortGUI)

23. Terakhir, pada method `main()`, jalankan GUI menggunakan `SwingUtilities.invokeLater()`. Setelah objek `MergeSortGUI_2511531016`

dibuat, tampilan aplikasi akan muncul dan siap digunakan oleh pengguna untuk melihat proses Merge Sort secara visual.

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater(() -> {  
        MergeSortGUI_2511531016 gui_1016 = new MergeSortGUI_2511531016();  
        gui_1016.setVisible(true);  
    });  
}
```

Gambar 2.66 (Langkah 23 MergeSortGUI)

2.4 Analisis Hasil

Berdasarkan hasil praktikum yang telah dilakukan, setiap algoritma sorting mampu mengurutkan data sesuai dengan tujuan yang ditetapkan. Shell Sort, Quick Sort, dan Merge Sort berhasil menghasilkan urutan data yang benar dari nilai terkecil ke terbesar. Perbedaannya terlihat pada cara masing-masing algoritma bekerja. Shell Sort melakukan pengurutan dengan membandingkan elemen yang memiliki jarak tertentu (gap), Quick Sort memanfaatkan pivot untuk membagi data menjadi beberapa bagian, sedangkan Merge Sort menggunakan metode divide and conquer dengan membagi array menjadi bagian yang lebih kecil lalu menggabungkannya kembali dalam keadaan terurut. Dari pengujian yang dilakukan, seluruh algoritma memberikan hasil akhir yang sama meskipun proses yang ditempuh berbeda.

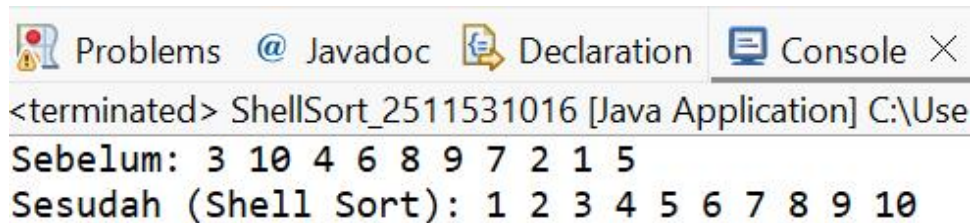
Pada implementasi Bubble Sort GUI dan Merge Sort GUI, proses pengurutan menjadi lebih mudah dipahami karena setiap langkah ditampilkan secara visual. Penggunaan warna pada elemen yang sedang diproses serta log langkah-langkah membantu memperlihatkan bagaimana algoritma bekerja secara bertahap, bukan hanya menampilkan hasil akhir. Melalui visualisasi tersebut dapat dilihat bahwa Bubble Sort melakukan banyak perbandingan antar elemen yang berdekatan, sedangkan Merge Sort lebih fokus pada proses penggabungan data yang telah dibagi sebelumnya. Dengan demikian, praktikum ini tidak hanya membantu memahami hasil pengurutan, tetapi juga memberikan gambaran yang lebih jelas mengenai mekanisme kerja masing-masing algoritma sorting.

BAB III

KESIMPULAN

3.1 Hasil Praktikum

3.1.1 ShellSort_2511531016

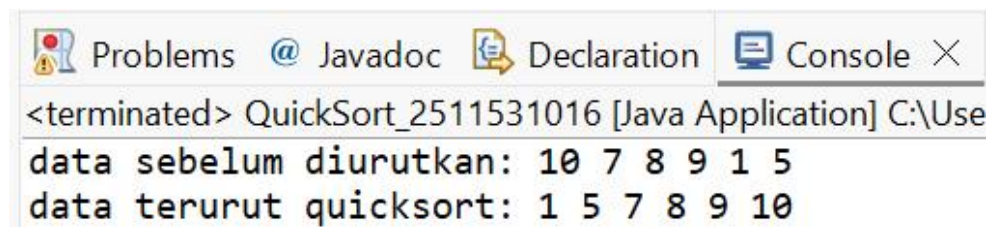


```
<terminated> ShellSort_2511531016 [Java Application] C:\Use
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

Gambar 3.1 (Hasil Praktikum ShellSort)

Output program menampilkan data awal berupa angka 3 10 4 6 8 9 7 2 1 5 berhasil diurutkan menjadi 1 2 3 4 5 6 7 8 9 10. Hal ini menunjukkan bahwa algoritma Shell Sort bekerja dengan baik dalam mengurutkan data secara ascending. Proses pengurutan dilakukan dengan membandingkan elemen yang memiliki jarak tertentu (gap), kemudian jarak tersebut diperkecil secara bertahap hingga akhirnya seluruh data tersusun secara berurutan dari nilai terkecil ke terbesar.

3.1.2 QuickSort_2511531016

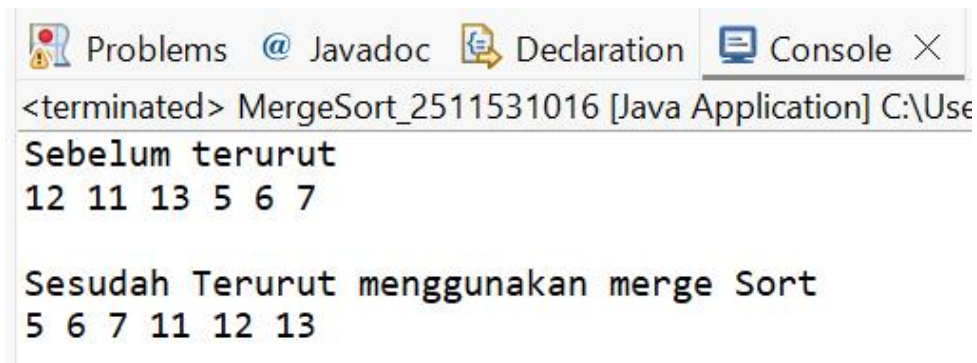


```
<terminated> QuickSort_2511531016 [Java Application] C:\Use
data sebelum diurutkan: 10 7 8 9 1 5
data terurut quicksort: 1 5 7 8 9 10
```

Gambar 3.2 (Hasil Praktikum QuickSort)

Output program menampilkan data awal 10 7 8 9 1 5 berhasil diurutkan menjadi 1 5 7 8 9 10. Pengurutan dilakukan dengan memilih sebuah nilai sebagai pivot, kemudian memisahkan elemen yang lebih kecil dan lebih besar dari pivot ke posisi yang sesuai. Setelah proses rekursif selesai pada setiap bagian array, seluruh data menjadi terurut secara ascending sehingga membuktikan bahwa implementasi Quick Sort berjalan dengan benar.

3.1.3 MergeSort_2511531016



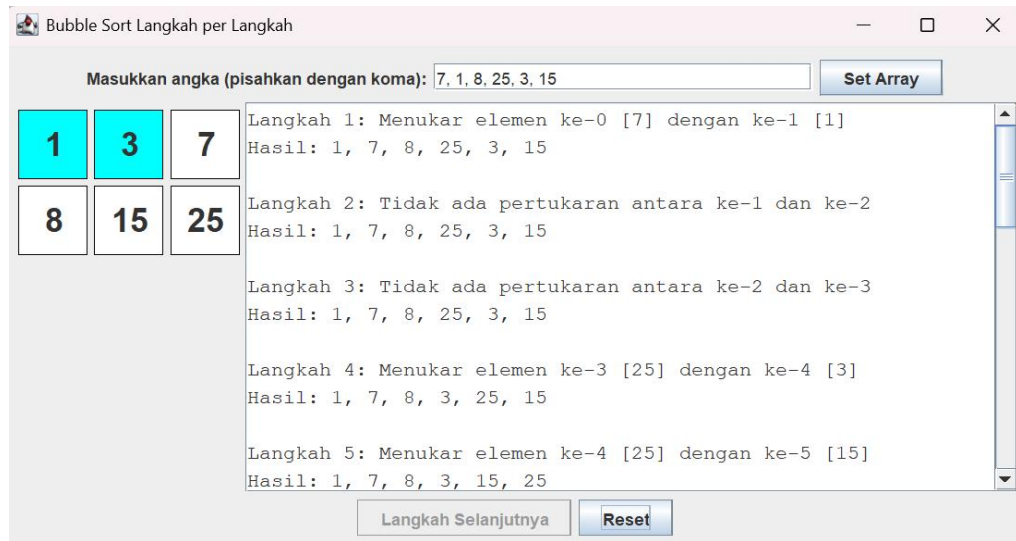
```
<terminated> MergeSort_2511531016 [Java Application] C:\Use
Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan merge Sort
5 6 7 11 12 13
```

Gambar 3.3 (Hasil Praktikum MergeSort)

Output program menampilkan data awal 12 11 13 5 6 7 berhasil diurutkan menjadi 5 6 7 11 12 13. Algoritma Merge Sort bekerja dengan membagi array menjadi beberapa bagian yang lebih kecil, kemudian mengurutkan setiap bagian tersebut sebelum menggabungkannya kembali menjadi satu array yang terurut. Hasil akhir memperlihatkan bahwa seluruh elemen telah tersusun dari nilai terkecil ke terbesar sesuai dengan tujuan pengurutan ascending.

3.1.4 BubbleSortGUI_2511531016

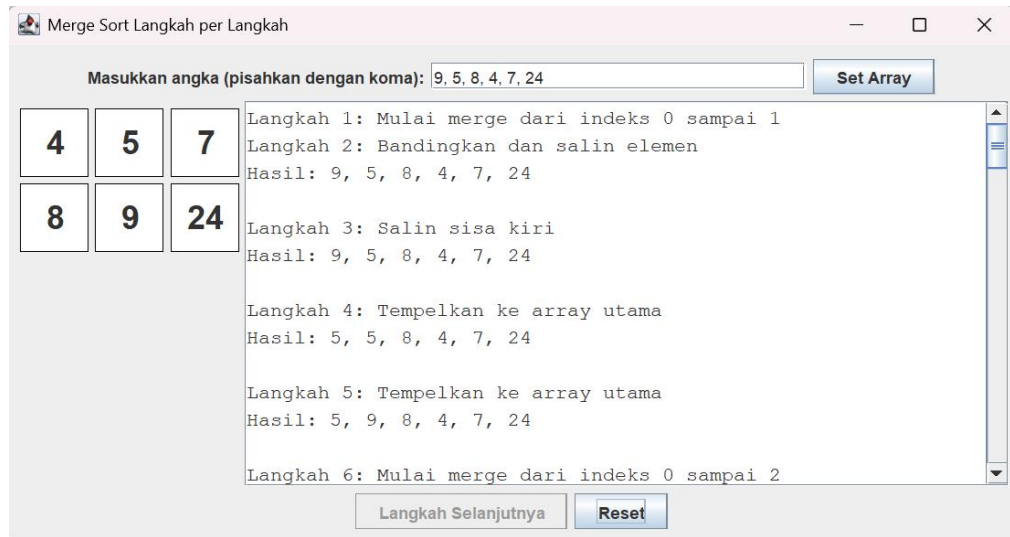


Gambar 3.4 (Hasil Praktikum BubbleSortGUI)

Output program di atas terlihat bahwa pengguna memasukkan data 7, 1, 8, 25, 3, 15 dan program menampilkan proses pengurutan secara bertahap. Setiap langkah menunjukkan perbandingan dua elemen yang berdekatan, apakah terjadi pertukaran atau tidak. Pada langkah pertama, angka 7 dan 1 ditukar karena 7 lebih besar dari 1, sedangkan pada langkah kedua dan ketiga tidak terjadi pertukaran karena elemen yang dibandingkan sudah berada pada urutan yang benar.

Warna biru pada kotak angka menunjukkan elemen yang sedang dibandingkan, sedangkan log di sebelah kanan menampilkan detail setiap langkah beserta kondisi array setelah proses dilakukan. Output ini menunjukkan cara kerja Bubble Sort yang menggeser nilai terbesar ke bagian kanan array melalui serangkaian perbandingan dan pertukaran elemen yang bersebelahan.

3.1.5 MergeSortGUI_2511531016



Gambar 3.4 (Hasil Praktikum MergeSortGUI)

Output program di atas terlihat bahwa pengguna memasukkan data 9, 5, 8, 4, 7, 24 dan program menampilkan proses penggabungan (merge) secara bertahap. Langkah pertama menunjukkan dimulainya proses merge pada indeks tertentu, kemudian program membandingkan elemen dari dua bagian array yang telah dipisahkan sebelumnya. Setelah elemen dibandingkan, nilai yang lebih kecil disalin ke array sementara, lalu ditempatkan kembali ke array utama hingga terbentuk urutan yang lebih teratur.

Log di sebelah kanan memperlihatkan setiap tahapan seperti membandingkan elemen, menyalin sisa elemen, dan menempelkan hasil merge ke array utama. Output ini menggambarkan prinsip kerja Merge Sort yang membagi data menjadi bagian-bagian kecil, kemudian menggabungkannya kembali dalam keadaan terurut sampai seluruh array tersusun secara ascending.

3.2 Saran Pengembangan

Program yang telah dibuat sudah mampu menjalankan proses pengurutan dengan baik, namun masih dapat dikembangkan agar lebih menarik dan informatif. Salah satu pengembangan yang dapat dilakukan adalah menambahkan fitur perhitungan jumlah perbandingan (comparison) dan jumlah pertukaran (swap) selama proses sorting berlangsung. Dengan adanya fitur tersebut, pengguna dapat membandingkan efisiensi masing-masing algoritma secara lebih jelas, terutama ketika jumlah data yang digunakan semakin besar.

Selain itu, tampilan GUI dapat ditingkatkan dengan menambahkan animasi pergerakan elemen dan pengaturan kecepatan proses sorting. Pengguna juga dapat diberikan pilihan untuk mengurutkan data secara ascending maupun descending serta memasukkan jumlah data yang lebih banyak secara dinamis. Pengembangan tersebut akan membuat program tidak hanya berfungsi sebagai alat demonstrasi algoritma, tetapi juga menjadi media pembelajaran yang lebih interaktif dan mudah dipahami.

DAFTAR PUSTAKA

- [1] A. H. Nasution dan R. Pratama, “Perbandingan Kinerja Algoritma Quick Sort, Merge Sort, dan Shell Sort pada Pengolahan Data Numerik,” *Jurnal Informatika dan Teknologi Komputer*, vol. 4, no. 2, pp. 87–95, 2023.
- [2] M. R. Saputra, Y. Firmansyah, dan D. Kurniawan, “Implementasi Algoritma Sorting pada Aplikasi Berbasis Java untuk Pembelajaran Struktur Data,” *Jurnal Sistem Informasi dan Komputer*, vol. 8, no. 1, pp. 25–33, 2024.
- [3] N. A. Putri dan F. Hidayat, “Visualisasi Proses Pengurutan Data Menggunakan Java Swing sebagai Media Pembelajaran Algoritma Sorting,” *Jurnal Teknologi Informasi dan Pendidikan*, vol. 6, no. 3, pp. 112–120, 2023.

TUGAS PRAKTIKUM PEKAN 8

Mengurutkan Playlist lagu

1. Deskripsi Tugas

Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (A-Z) atau Durasi (terpendek ke terpanjang)

2. Kode Program

2.1 Sorting_2511531016

```
package pekan8_2511531016;
import java.util.Scanner;

public class Sorting_2511531016 {
    static class Lagu_1016 {
        String judul_1016;
        String penyanyi_1016;
        int durasi_1016;

        public Lagu_1016(String judul_1016, String penyanyi_1016, int durasi_1016) {
            this.judul_1016 = judul_1016;
            this.penyanyi_1016 = penyanyi_1016;
            this.durasi_1016 = durasi_1016;
        }
    }

    static Lagu_1016[] buatDataLagu_1016() {
        return new Lagu_1016[] {
            new Lagu_1016("Sunday Morning", "Maroon 5", 244),
            new Lagu_1016("About You", "The 1975", 326),
            new Lagu_1016("Drop Dead", "Olivia Rodrigo", 224),
            new Lagu_1016("Bukan Orangnya", "Juicy Luicy", 190),
            new Lagu_1016("Monolog", "Pamungkas", 207),
            new Lagu_1016("Dulu Kita Masih Remaja", "The Panasdalam Bank", 189),
            new Lagu_1016("Yang Terdalam", "Noah", 199)
        };
    }

    static void tampilData_1016(Lagu_1016[] dataLagu_1016) {
        for (int i_1016 = 0; i_1016 < dataLagu_1016.length; i_1016++) {
            System.out.println((i_1016 + 1) + ". " + dataLagu_1016[i_1016].judul_1016 + " - " + dataLagu_1016[i_1016].durasi_1016 + " detik");
        }
    }

    static void shellSort_1016(Lagu_1016[] dataLagu_1016) {
        int n_1016 = dataLagu_1016.length;
        for (int gap_1016 = n_1016 / 2; gap_1016 > 0; gap_1016 /= 2) {
            for (int i_1016 = gap_1016; i_1016 < n_1016; i_1016++) {
                Lagu_1016 temp_1016 = dataLagu_1016[i_1016];
                int j_1016 = i_1016;
                while (j_1016 >= gap_1016 && dataLagu_1016[j_1016 - gap_1016].judul_1016.compareToIgnoreCase(temp_1016.judul_1016) > 0) {
                    dataLagu_1016[j_1016] = dataLagu_1016[j_1016 - gap_1016];
                    j_1016 -= gap_1016;
                }
                dataLagu_1016[j_1016] = temp_1016;
            }
        }
    }

    static void swap_1016(Lagu_1016[] dataLagu_1016, int i_1016, int j_1016) {
        Lagu_1016 temp_1016 = dataLagu_1016[i_1016];
        dataLagu_1016[i_1016] = dataLagu_1016[j_1016];
        dataLagu_1016[j_1016] = temp_1016;
    }

    static int partition_1016(Lagu_1016[] dataLagu_1016, int low_1016, int high_1016) {
        int pivot_1016 = dataLagu_1016[high_1016].durasi_1016;
        int i_1016 = low_1016 - 1;
        for (int j_1016 = low_1016; j_1016 < high_1016; j_1016++) {
            if (dataLagu_1016[j_1016].durasi_1016 < pivot_1016) {
                i_1016++;
                swap_1016(dataLagu_1016, i_1016, j_1016);
            }
        }
        swap_1016(dataLagu_1016, i_1016 + 1, high_1016);
        return i_1016 + 1;
    }
}
```

```

static void quickSort_1016(Lagu_1016[] dataLagu_1016, int low_1016, int high_1016) {
    if (low_1016 < high_1016) {
        int pi_1016 = partition_1016(dataLagu_1016, low_1016, high_1016);
        quickSort_1016(dataLagu_1016, low_1016, pi_1016 - 1);
        quickSort_1016(dataLagu_1016, pi_1016 + 1, high_1016);
    }
}

static void merge_1016(Lagu_1016[] dataLagu_1016, int left_1016, int mid_1016, int right_1016) {
    int n1_1016 = mid_1016 - left_1016 + 1;
    int n2_1016 = right_1016 - mid_1016;
    Lagu_1016[] L_1016 = new Lagu_1016[n1_1016];
    Lagu_1016[] R_1016 = new Lagu_1016[n2_1016];
    for (int i_1016 = 0; i_1016 < n1_1016; i_1016++) {
        L_1016[i_1016] = dataLagu_1016[left_1016 + i_1016];
    }
    for (int j_1016 = 0; j_1016 < n2_1016; j_1016++) {
        R_1016[j_1016] = dataLagu_1016[mid_1016 + 1 + j_1016];
    }
    int i_1016 = 0;
    int j_1016 = 0;
    int k_1016 = left_1016;
    while (i_1016 < n1_1016 && j_1016 < n2_1016) {
        if (L_1016[i_1016].judul_1016.compareToIgnoreCase(R_1016[j_1016].judul_1016) <= 0) {
            dataLagu_1016[k_1016] = L_1016[i_1016];
            i_1016++;
        } else {
            dataLagu_1016[k_1016] = R_1016[j_1016];
            j_1016++;
        }
        k_1016++;
    }
    while (i_1016 < n1_1016) {
        dataLagu_1016[k_1016] = L_1016[i_1016];
        i_1016++;
        k_1016++;
    }
    while (j_1016 < n2_1016) {
        dataLagu_1016[k_1016] = R_1016[j_1016];
        j_1016++;
        k_1016++;
    }
}

static void mergeSort_1016(Lagu_1016[] dataLagu_1016, int left_1016, int right_1016) {
    if (left_1016 < right_1016) {
        int mid_1016 = (left_1016 + right_1016) / 2;
        mergeSort_1016(dataLagu_1016, left_1016, mid_1016);
        mergeSort_1016(dataLagu_1016, mid_1016 + 1, right_1016);
        merge_1016(dataLagu_1016, left_1016, mid_1016, right_1016);
    }
}

public static void main(String[] args) {
    Scanner input_1016 = new Scanner(System.in);
    Lagu_1016[] dataLagu_1016 = buatDataLagu_1016();
    System.out.println("=== Sorting Playlist NIM: 2511531016 ===");
    System.out.print("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
    int pilih_1016 = input_1016.nextInt();
    System.out.println("\nData Sebelum Sorting:");
    tampilData_1016(dataLagu_1016);

    switch (pilih_1016) {
        case 1:
            shellSort_1016(dataLagu_1016);
            System.out.println("\nData Setelah Shell Sort (Judul A-Z):");
            tampilData_1016(dataLagu_1016);
            break;

        case 2:
            quickSort_1016(dataLagu_1016, 0, dataLagu_1016.length - 1);
            System.out.println("\nData Setelah Quick Sort (Durasi Asc):");
            tampilData_1016(dataLagu_1016);
            break;

        case 3:
            mergeSort_1016(dataLagu_1016, 0, dataLagu_1016.length - 1);
            System.out.println("\nData Setelah Merge Sort (Judul A-Z):");
            tampilData_1016(dataLagu_1016);
            break;

        default:
            System.out.println("\nPilihan tidak valid!");
    }
    input_1016.close();
}

```

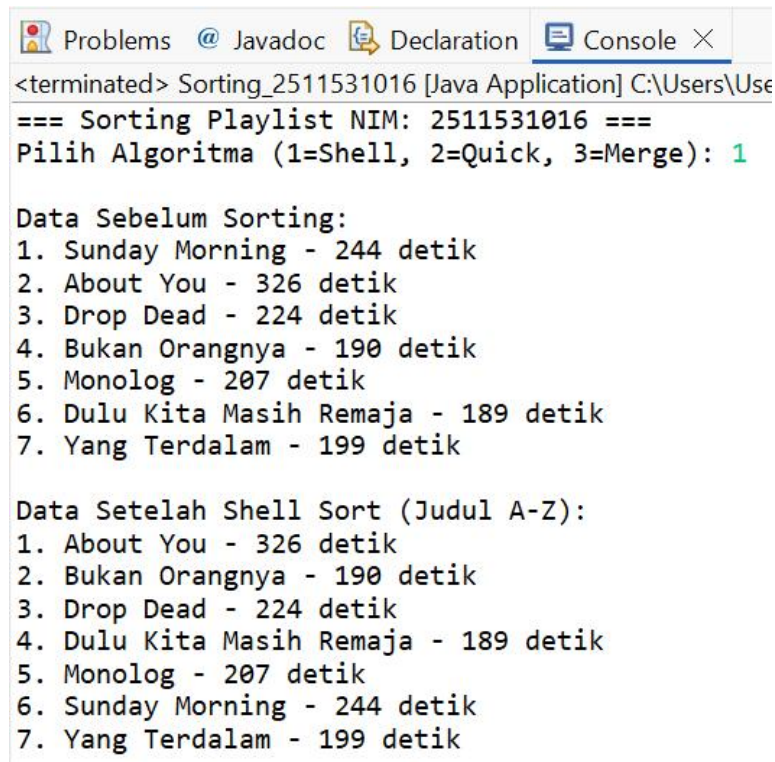

Penjelasan :

Kode program `Sorting_2511531016` diawali dengan pembuatan class `Sorting_2511531016` sebagai class utama yang berisi seluruh proses pengolahan data. Di dalam class tersebut terdapat class `Lagu_1016` yang digunakan untuk menyimpan informasi setiap lagu, yaitu judul lagu, nama penyanyi, dan durasi lagu. Selanjutnya dibuat method `buatDataLagu_1016()` untuk menginisialisasi data playlist yang akan digunakan pada proses pengurutan. Data yang telah dibuat kemudian dapat ditampilkan menggunakan method `tampilData_1016()`, sehingga pengguna dapat melihat isi playlist sebelum maupun sesudah proses sorting dilakukan.

Proses pengurutan data dilakukan menggunakan tiga algoritma yang berbeda, yaitu Shell Sort, Quick Sort, dan Merge Sort. Method `shellSort_1016()` digunakan untuk mengurutkan data berdasarkan judul lagu secara alfabetis dengan memanfaatkan nilai gap yang semakin kecil pada setiap iterasi. Method `quickSort_1016()` digunakan untuk mengurutkan data berdasarkan durasi lagu menggunakan teknik pembagian data terhadap nilai pivot. Sementara itu, method `mergeSort_1016()` bekerja dengan membagi data menjadi beberapa bagian yang lebih kecil, kemudian menggabungkannya kembali dalam keadaan terurut. Setiap algoritma dibuat dalam method terpisah sehingga program menjadi lebih terstruktur dan mudah dipahami.

Pada method `main()`, program akan menampilkan menu pilihan algoritma sorting yang dapat digunakan oleh pengguna. Setelah pengguna memasukkan pilihan, data playlist ditampilkan terlebih dahulu sebagai data awal. Program kemudian menjalankan algoritma yang dipilih dan menampilkan hasil pengurutan sesuai metode yang digunakan. Dengan adanya menu tersebut, pengguna dapat membandingkan hasil pengurutan dari beberapa algoritma dalam satu program yang sama serta memahami penerapan konsep sorting pada data objek lagu.

3. Output Kode Program



```
<terminated> Sorting_2511531016 [Java Application] C:\Users\Use
=== Sorting Playlist NIM: 2511531016 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 1

Data Sebelum Sorting:
1. Sunday Morning - 244 detik
2. About You - 326 detik
3. Drop Dead - 224 detik
4. Bukan Orangnya - 190 detik
5. Monolog - 207 detik
6. Dulu Kita Masih Remaja - 189 detik
7. Yang Terdalam - 199 detik

Data Setelah Shell Sort (Judul A-Z):
1. About You - 326 detik
2. Bukan Orangnya - 190 detik
3. Drop Dead - 224 detik
4. Dulu Kita Masih Remaja - 189 detik
5. Monolog - 207 detik
6. Sunday Morning - 244 detik
7. Yang Terdalam - 199 detik
```

Penjelasan :

Pada output program terlihat bahwa pengguna memilih algoritma Shell Sort dengan memasukkan angka 1 pada menu pilihan. Program kemudian menampilkan data playlist sebelum dilakukan proses pengurutan. Data awal masih berada dalam urutan sesuai saat disimpan ke dalam array, yaitu terdiri dari tujuh judul lagu dengan durasi yang berbeda-beda. Tampilan ini digunakan sebagai data pembanding untuk melihat perubahan urutan setelah proses sorting dilakukan.

Setelah Shell Sort dijalankan, program berhasil mengurutkan data berdasarkan judul lagu dari A sampai Z. Algoritma Shell Sort bekerja dengan membandingkan elemen-elemen yang memiliki jarak tertentu (gap), kemudian jarak tersebut diperkecil secara bertahap hingga menjadi satu. Dengan cara ini, data dapat bergerak lebih cepat menuju posisi yang benar dibandingkan metode pertukaran sederhana. Hasil output menunjukkan bahwa judul lagu telah tersusun

secara alfabetis mulai dari "About You" hingga "Yang Terdalam", sedangkan durasi setiap lagu tetap tidak berubah. Hal ini membuktikan bahwa implementasi Shell Sort pada program telah berjalan dengan baik dan menghasilkan urutan data yang sesuai dengan kriteria yang ditentukan.

4. Penjelasan Pilihan Algoritma dan Kompleksitas

Pada tugas praktikum ini dipilih algoritma Shell Sort untuk mengurutkan data playlist lagu berdasarkan judul lagu secara alfabetis. Shell Sort merupakan pengembangan dari Insertion Sort yang menggunakan nilai gap (jarak) untuk membandingkan elemen-elemen yang posisinya berjauhan. Pada program yang dibuat, nilai gap diawali dengan setengah dari jumlah data, kemudian diperkecil secara bertahap hingga bernilai 1. Dengan cara ini, data dapat bergerak lebih cepat menuju posisi yang seharusnya sehingga proses pengurutan menjadi lebih efisien dibandingkan Insertion Sort biasa.

Algoritma Shell Sort memiliki kompleksitas waktu terbaik sebesar $O(n \log n)$, sedangkan kompleksitas rata-ratanya bergantung pada urutan gap yang digunakan dan umumnya berada di antara $O(n \log^2 n)$ hingga $O(n^{1.5})$. Untuk kondisi terburuk, kompleksitas waktunya adalah $O(n^2)$. Selain itu, Shell Sort memiliki kompleksitas ruang $O(1)$ karena proses pengurutan dilakukan langsung pada array tanpa memerlukan struktur data tambahan yang besar.

Pemilihan Shell Sort pada program ini didasarkan pada proses implementasinya yang relatif sederhana serta kemampuannya dalam melakukan pengurutan dengan lebih cepat dibandingkan algoritma sorting dasar seperti Bubble Sort dan Insertion Sort. Untuk jumlah data playlist yang tidak terlalu besar, Shell Sort dapat memberikan hasil pengurutan yang baik dengan penggunaan memori yang tetap efisien.